CSE 396 – Spring 2025
Module Requirements-Documentation



SD Belt

(Scan & Detect Belt)

Group 15 a.k.a. “Belt Innovations”

25.04.2025

Belt Innovations	Image Processing & AI	Hardware	Mobile Application	Desktop Application	Database and Backend
Ahmet Abdulgaffar Fettahoğlu			+		+
Büşra Sarı	+				+
Cemil Fatih Yol	+		+		
Kağan Çakıroğlu		+		+	
Mehmet Hayrullah Özkul			+		+
Mert Emir Şeker		+		+	
Osmancan Bozali			+		+
Serkan Efe Çamoğlu	+			+	
Ziya Kadir Tokluoğlu	+	+			

Table of Contents

Modules 5

1. **YOLO-Based Image Processing Module** 5
   - 1.1. **Introduction** 5
   - 1.2. **Technology Approach** 5
   - 1.3. **Dataset and Data Handling** 6
   - 1.4. **Preprocessing & Postprocessing** 6
   - 1.5. **Model Training and Fine-Tuning** 7
   - 1.6. **Deployment on Raspberry Pi** 7
   - 1.7. **Future Improvements** 7
   - 1.8. **Hardware-Camera Communication System** 8
   - 1.9. **API Integration for Data Transmission** 10
   - 1.10. **Test Cases for YOLO-Based Image Processing Module** 12
2. **Desktop Application Module** 14
   - 2.1. **Overview** 14
   - 2.2. **Implemented Features** 14
   - 2.3. **Possible Future Features** 15
   - 2.4. **Technologies Used** 15
   - 2.5. **Message Format Examples** 15
   - 2.6. **Desktop Figma Designs** 16
   - 2.7. **Test Cases for Desktop Application Module** 18
3. **Hardware Module** 20
   - 3.1. **Overview** 20
   - 3.2. **SD Belt Videos** 20
   - 3.3. **SD Belt Workflow** 22
   - 3.4. **Test Cases for Hardware Module** 23
4. **Mobile Application Module** 25
   - 4.1. **Introduction** 25
   - 4.2. **Technology Approach** 25
   - 4.3. **Current Progress** 26
   - 4.4. **Development Roadmap** 28
   - 4.5. **Integration with Other Modules** 29
   - 4.6. **Test Cases for Mobile Application Module** 29
5. **Database and Backend Module** 30
   - 5.1. **Introduction** 30
   - 5.2. **Technology Approach** 30
   - 5.3. **Backend Architecture** 30

5.4. Development Roadmap.....31

5.5. Integration with Other Modules The backend coordinates interactions across all SD Belt modules:.....32

5.6. Test Cases for Database and Backend Module.....32

Modules

1. YOLO-Based Image Processing Module

1.1. Introduction

This section details the implementation of the YOLOv8n model for our automated defect detection system in the manufacturing process. Building upon our previous work with the hardware-camera communication system, this report focuses on the integration of the YOLOv8n model in a C++ environment on our Raspberry Pi hardware platform with the Hailo 13tdos AI acceleration module.

The defect detection system represents a critical quality control mechanism designed to identify anomalous products on the manufacturing line in real-time. By leveraging advanced computer vision techniques through the YOLOv8n architecture, we aim to achieve high accuracy detection with minimal processing latency.

1.2. Technology Approach

1.2.1. Technology Stack

- **Hardware**: Raspberry Pi 4, Standard USB webcams (cost-efficient alternative to industrial cameras)
- **AI Framework**: YOLOv8n (Ultralytics Nano version, optimized for edge devices like Raspberry Pi)
- **Language**: C++ for model execution and system control
- **Image Processing Library**: OpenCV (for preprocessing, masking, and visualization)

1.2.2. Layered Detection Architecture

The detection system will follow a two-stage pipeline:

- **Fruit Detection Layer**: Identify all fruits in the frame.
- **Defect Detection Layer**: Detect and localize defects within each identified fruit using bounding boxes.

This layered architecture enables multi-fruit handling and refined defect localization.

1.3. Dataset and Data Handling

1.3.1. Dataset Collection

We will use a combination of publicly available datasets and a custom-generated dataset. Public datasets will provide a base level of variety, while our custom dataset will focus on specific fruit types and defect scenarios relevant to our setup:

- Capture images of multiple fruit types under various conditions.
- Adjust lighting and background to assist model learning.
- Simulate and label real-world defect conditions.

1.3.2. Data Augmentation

To improve generalization and robustness:

- Rotation and flipping
- Brightness and contrast adjustments
- Gaussian noise and blur
- Occlusion simulation (partial covering)

1.4. Preprocessing & Postprocessing

1.4.1. Preprocessing

- **Masking**: Apply masks to isolate fruits from the background using color thresholding and contour analysis.
- **Resizing**: Resize images to YOLOv8n's expected input size (e.g., 640x640).
- **Normalization**: Normalize pixel values according to model input requirements.

1.4.2. Postprocessing

- **Bounding Box Filtering**: Filter predictions based on confidence threshold.
- **Defect Highlighting**: Overlay bounding boxes around detected defect regions.
- **Defect Area Calculation**: Estimate the defect area as a percentage of each fruit’s bounding box for reporting.

1.5. Model Training and Fine-Tuning

1.5.1. Training Strategy

- Use YOLOv8n pretrained weights.
- Fine-tune the model using a combination of public and custom datasets.
- Incorporate diverse scenarios to improve multi-fruit detection and defect localization.

1.5.2. Hyperparameter Tuning

- Optimize learning rate, batch size, and training epochs.
- Refine IOU thresholds and anchor boxes (if needed) to boost accuracy.

1.5.3. Evaluation Metrics

- Precision, Recall, and mAP (mean Average Precision)
- Real-time performance benchmarks on Raspberry Pi

1.6. Deployment on Raspberry Pi

1.6.1. Optimization

- Use YOLOv8n for efficient edge deployment.
- Leverage OpenCV-DNN or Ultralytics runtime optimized for Raspberry Pi.

1.6.2. Real-Time Pipeline

- Capture frames using 2–4 connected cameras
- Preprocess frames using OpenCV
- Run YOLO inference
- Postprocess and visualize results or trigger actuators

1.7. Implementation Overview

1.7.1. YOLOv8n Model Architecture Overview

1.7.1.1. Model Selection Rationale

The YOLOv8n (nano) variant was selected for our implementation due to its optimal balance between:

- **Inference Speed:** Critical for real-time processing on resource-constrained hardware
- **Detection Accuracy:** Sufficient precision for manufacturing defect identification
- **Resource Requirements:** Compatibility with our Raspberry Pi hardware constraints
- **ONNX Compatibility:** Ability to export to ONNX format for cross-platform deployment

YOLOv8n represents the lightest weight variant of the YOLOv8 family while maintaining acceptable accuracy metrics for our specific use case. The model's architecture is optimized

for efficiency, making it suitable for deployment on edge devices like our Raspberry Pi setup.

1.7.1.2. Technical Specifications

YOLOv8n offers the following technical capabilities relevant to our implementation:

Parameter	Specification
Input Resolution	640×640 pixels
Model Size	Approximately 6.2 MB
GFLOPs	8.7
Parameters	3.2M
Inference Speed (Raspberry Pi)	Target: 5-10 FPS
mAP (COCO val)	37.3%

The compact nature of YOLOv8n makes it particularly well-suited for our deployment scenario, where processing resources are limited but real-time performance is essential.

1.7.2. Defect Detection Strategy

1.7.2.1. Detection Methodology

Our defect detection system employs an anomaly detection approach using YOLOv8n as follows:

- **Reference Model Training**: The YOLOv8n model is trained on a dataset of correctly manufactured products, establishing a baseline for "normal" product appearance.
- **Anomaly Threshold Determination**: Statistical analysis of model outputs on known good products establishes confidence thresholds that differentiate between normal product variations and actual defects.
- **Classification Approach**: Rather than training on specific defect types, our implementation focuses on detecting deviation from expected product standards, allowing for identification of novel or unexpected defects.
- **Multi-Angle Analysis**: By utilizing frames from multiple camera angles (as detailed in our hardware-camera communication system), the model can detect defects that might only be visible from certain perspectives.

1.7.3. C++ Implementation Architecture

The YOLOv8n model is implemented within a C++ framework to ensure optimal performance. The implementation follows this structure:

```
├── include/
│   ├── yolo_detector.h      // YOLOv8n detector class declaration
│   ├── frame_processor.h    // Interface for processing camera frames
├── src/
│   ├── yolo_detector.cpp    // Implementation of YOLOv8n detector
│   ├── frame_processor.cpp  // Frame processing implementation
└── models/
    └── yolov8n.onnx         // YOLOv8n model in ONNX format
```

This architecture separates concerns between model inference, frame processing, and hardware acceleration, providing a modular approach that facilitates debugging and optimization.

1.7.4. Frame Pre-processing pipeline

Before passing frames to the YOLOv8n model, our C++ implementation performs the following pre-processing steps:

- **Resolution Adjustment**: Resizing input frames to 640×640 pixels to match YOLOv8n input requirements
- **Normalization**: Pixel values normalized to range [0,1]
- **Channel Ordering**: Conversion from BGR (OpenCV default) to RGB colour space
- **Tensor Formatting**: Conversion of processed image to appropriate tensor format for model input

These pre-processing steps are implemented as an efficient pipeline to minimize latency between frame capture and inference.

1.7.5. Inference and Post-processing

The inference and post-processing stages include:

- **Model Inference**: Execution of the YOLOv8n model on the pre-processed frame
- **Bounding Box Extraction**: Parsing model output to extract bounding box coordinates
- **Confidence Filtering**: Removal of detections below confidence threshold
- **Non-Maximum Suppression**: Elimination of redundant overlapping detections
- **Anomaly Determination**: Analysis of detection results to determine if a defect is present
- **Result Annotation**: Marking detected defects on output frames for visualization

1.7.6. Integration with Thread Architecture

1.7.6.1. Thread-Based Processing Model

Building on our existing thread architecture detailed in the previous report, the YOLOv8n implementation is integrated as follows:

- **Camera Acquisition Threads**: Dedicated threads for each camera (two at 180-degree angles, with optional third top-down camera)

- **Frame Queue Management**: Thread-safe queue system for passing captured frames to processing threads
- **YOLOv8n Inference Thread**: Dedicated thread for running YOLOv8n inference on the Hailo AI acceleration module
- **Result Processing Thread**: Thread for post-processing detection results and triggering appropriate actions
  This multi-threaded architecture enables parallel operation of:
  - Frame acquisition from multiple cameras
  - YOLOv8n model inference
  - Post-processing of detection results
  - Visualization and control system interaction

1.7.7. Data Flow Architecture

The data flow within our YOLOv8n implementation follows this sequence:

Camera Threads → Frame Queue → Pre-processing → YOLOv8n Inference → Post-processing → Action Trigger

↑
|

Hailo AI Acceleration

This architecture maintains continuous frame capture even when processing experiences temporary delays, ensuring no product passes the inspection zone without analysis.

1.7.8. Thread Synchronization Strategy

To manage concurrent access to shared resources, our implementation employs:

- Mutex Locks: For thread-safe access to shared data structures
- Condition Variables: For signaling between threads when new frames are available
- Atomic Operations: For thread-safe counters and flags
- Lock-Free Queues: For high-performance frame passing between threads

These synchronization mechanisms are carefully implemented to minimize overhead while ensuring thread safety.

1.7.9. Integration with Camera System

1.7.9.1. Camera-Model Interface

The YOLOv8n model interfaces with our multi-camera system as follows:

- **Frame Selection Logic**: Algorithm to determine which camera frames to process based on conveyor belt timing and product position
- **Multi-View Analysis**: Capability to analyze multiple views of the same product to detect defects visible from specific angles
- **Temporal Analysis**: Optional tracking of products across multiple frames to improve detection reliability

1.7.9.2. Camera Configuration Requirements

The YOLOv8n model implementation requires specific camera configurations:

- **Resolution**: Minimum 640×480 resolution (frames are resized to 640×640 for inference)
- **Frame Rate**: Minimum 15 FPS to ensure adequate coverage of moving products
- **Exposure Settings**: Configured for consistent lighting conditions to reduce false positives
- **Focus Settings**: Fixed focus to maintain consistent image quality

1.7.10. ONNX Format Implementation

1.7.10.1. ONNX Conversion Process

The YOLOv8n model is converted to ONNX format through the following process:

- **Model Training**: Initial training in PyTorch using the Ultralytics YOLOv8 framework
- **Export Conversion**: Conversion to ONNX format using Ultralytics export functionality
- **Optimization**: ONNX model optimization for inference speed using ONNX Runtime
- **Quantization**: Optional int8 quantization for improved performance on the Hailo hardware

The resulting ONNX model provides a standardized format that can be deployed across different runtime environments.

1.7.10.2. ONNX Runtime Integration

Our C++ implementation integrates with ONNX Runtime through:

- **Session Management**: Creation and management of ONNX Runtime inference sessions
- **Memory Allocation**: Efficient buffer management for input and output tensors
- **Inference Execution**: Optimized execution of the YOLOv8n model on available hardware
- **Error Handling**: Robust error detection and recovery mechanisms

1.7.11. Hailo 13tdos AI Module Integration

1.7.11.1. Hailo Architecture Overview

The Hailo 13tdos AI module provides hardware acceleration for neural network inference with these key components:

- **Neural Network Core**: Specialized processors for efficient matrix operations
- **Memory Architecture**: Optimized for AI workload data flow
- **I/O Interfaces**: High-speed connections to the Raspberry Pi host

1.7.11.2. Hailo-ONNX Interface

Our implementation bridges ONNX and Hailo through:

- **Model Compilation**: Conversion of ONNX model to Hailo-compatible format
- **Runtime API**: C++ interface for inference execution on Hailo hardware
- **Buffer Management**: Efficient memory handling between host and Hailo device

1.7.12. Current Implementation Challenges

Despite the comprehensive architecture design, we are currently facing several significant implementation challenges:

1.7.12.1. YOLOv8n ONNX Format Compatibility Issues

The primary challenge we've encountered involves compatibility issues between the YOLOv8n ONNX model and the Hailo 13tdos AI module:

- **Model Conversion Errors**: The exported YOLOv8n ONNX model presents compatibility issues with the Hailo hardware compiler, resulting in conversion failures.
- **Operator Support Limitations**: Certain operators used in the YOLOv8n architecture are not fully supported by the current Hailo SDK, preventing successful compilation.
- **Input Shape Requirements**: Discrepancies between the expected input tensor shapes between our ONNX model and the Hailo hardware requirements.
- **Layer Compatibility**: Some layers in the YOLOv8n architecture are not efficiently mapped to the Hailo hardware architecture.

1.7.12.2. Raspberry Pi Camera Configuration Challenges

Additionally, we are encountering issues with the Raspberry Pi camera configuration:

- **Frame Rate Limitations**: Difficulty achieving the target frame rate (15+ FPS) necessary for effective real-time monitoring of the production line.
- **Multi-Camera Synchronization**: Challenges in synchronizing multiple camera feeds for coherent multi-angle analysis.
- **USB Bandwidth Constraints**: The Raspberry Pi's USB controller bandwidth limitations affect simultaneous operation of multiple high-resolution cameras.
- **Driver Compatibility**: Inconsistencies between camera drivers and the Raspberry Pi operating system causing intermittent connection issues.

1.7.12.3. Integration Challenges

The integration between different system components presents additional challenges:

- **Memory Management**: Efficient allocation and management of memory resources between camera frames, pre-processing, and the Hailo accelerator.
- **Timing Synchronization**: Ensuring consistent timing between frame capture, processing, and result evaluation for moving products.
- **Resource Contention**: Managing CPU and memory resources between camera operations and model inference.

1.7.13. Implementation Status

The current implementation status can be summarized as follows:

- **Camera System**: Multi-threaded camera acquisition framework implemented and operational at reduced frame rates.
- **Threading Architecture**: Core thread infrastructure operational with proven communication between threads.
- **YOLOv8n Model**: Successfully exported to ONNX format but encountering compatibility issues with Hailo hardware.
- **Pre/Post-processing**: Image pre-processing and result post-processing pipelines implemented and validated.
- **Hailo Integration**: Initial integration with Hailo SDK established but blocked by model compatibility issues.

The development team is currently focused on resolving the ONNX format compatibility issues with the Hailo 13tdos AI module while continuing to optimize the camera configuration to improve frame rates and stability.

1.7.14. Conclusion

The YOLOv8n implementation for our defect detection system represents a sophisticated integration of computer vision, multi-threaded programming, and hardware acceleration. While we have established a solid architectural foundation, the current implementation challenges with the YOLOv8n ONNX format on the Hailo 13tdos AI module and Raspberry Pi camera configurations have temporarily impeded full deployment.

The modular design of our system architecture will facilitate targeted resolution of these challenges while maintaining progress on other system components. Once these technical obstacles are overcome, the system promises to deliver an efficient, accurate, and cost-effective solution for automated defect detection in the manufacturing process.

1.1. Future Improvements

- Expand dataset with more fruit types and defect conditions
- Integrate centralized cloud storage and update system
- Add support for multiple processing nodes and camera stitching

This project sets the foundation for a scalable, cost-effective visual quality control solution using embedded AI.

1.2. Hardware-Camera Communication System

1.2.1. Key Responsibilities:

Multi-Camera Setup and Configuration:

- Establish and maintain connections with two webcams positioned strategically around the conveyor belt.
- Configure camera parameters such as resolution and frame rate to optimize image quality for defect detection.

Thread-Based Parallel Image Acquisition:

- Implement a multi-threaded architecture that dedicates independent threads to each camera.
- Ensure cameras can operate simultaneously to capture different angles of products on the conveyor belt.
- Address current low frame rate issues to improve overall system performance.

Image Acquisition Pipeline:

- Develop efficient frame capture and transfer mechanisms compatible with the YOLOv8n model requirements.
- Prepare captured images for processing by the defect detection system.

1.2.2. Implementation Details:

Camera Selection Rationale: Standard webcams are chosen over industrial cameras due to their cost efficiency while still providing adequate image quality for defect detection.

Camera Deployment Strategy:

- **Initial Configuration**: Two webcams are deployed at a 180-degree angle from each other, positioned on directly opposite sides of the conveyor belt. This arrangement ensures complete side coverage of products as they move along the belt.
- **Third Camera Option**: If necessary, a third camera will be mounted directly above the conveyor belt to provide a top-down view, creating a comprehensive three-dimensional capture system.

1.2.3. Performance Optimization Strategy:

a. **Current Status**: Currently, we are experiencing low frames-per-second (FPS) from the Raspberry Pi cameras, which is a limitation we're actively addressing.

b. **Thread-Based Solution**: Our multi-threaded architecture separates camera operations from processing, allowing cameras to capture frames continuously while defect detection occurs in parallel.

1.2.4. Thread Architecture Strategy:

The system implements a multi-threaded approach for camera management with these key features:

- **Dedicated Camera Threads**: Each webcam operates in its own dedicated thread, allowing simultaneous frame capture from different angles around the conveyor belt.
- **180-Degree Camera Positioning**: Two primary cameras are positioned at 180 degrees from each other on opposite sides of the conveyor belt, providing complete side views of products.
- **Optional Top Camera**: If necessary, a third camera positioned above the conveyor belt provides a top-down view, completing the visual coverage of products.
- **Centralized Processing**: A dedicated processing thread handles the YOLOv8n model execution on the Raspberry Pi AI Kit, keeping intensive processing separate from frame capture.
- **Continuous Operation**: The thread architecture ensures that frame capture continues uninterrupted even during intensive processing operations.

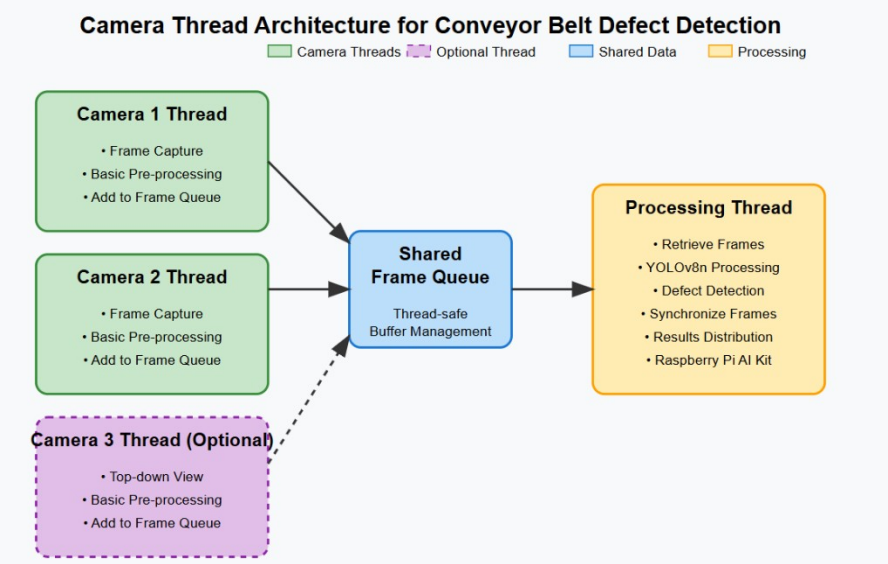


Figure 1: Thread Architecture for Multi-Camera Defect Detection System

1.2.5. Real-time Acquisition Approach:

- **Current Status**: Currently, we are experiencing low frames-per-second (FPS) from the Raspberry Pi cameras, which we aim to improve through our thread architecture.
- **Camera Positioning**: Two cameras positioned on opposite sides of the conveyor belt, with an optional third camera above for complete product coverage.
- **Target Frame Rate**: Working toward achieving higher frame rates by optimizing the thread structure and utilizing the Raspberry Pi AI Kit.

After processing images using the YOLO model in C++ to detect defective products on the production line, the resulting data must be transmitted to various parts of the system such as the desktop application, mobile app, and backend database.

1.3. API Integration for Data Transmission

1.3.1. Introduction

After processing images using the YOLO model in C++ to detect defective products on the production line, the resulting data must be transmitted to various parts of the system such as the desktop application, mobile app, and backend database.

1.3.2. Data Formatting and Communication

- **Data Generation:**
  The image processing unit produces detection results including:
    - Product ID
    - Detection timestamp
    - Defect presence
    - Confidence scores
    - Bounding box coordinates for the detected defects
- **Data Serialization:**
  The detection results are organized into a JSON object.
    - JSON is chosen for its language-agnostic properties and ease of integration, making it an ideal format to bridge communication among diverse system modules.

1.3.3. Core API Functionality

The SD Belt API serves as a central communication hub with these primary responsibilities:

1.3.3.1. Data Collection and Formatting

The API receives structured detection data from the YOLO processing module, including detailed metrics such as:

- Product identifiers
- Precise detection timestamps
- Binary defect presence indicators
- Confidence scores for statistical analysis
- Exact bounding box coordinates for defect visualization
- Defect area percentage calculations
    - Defect type classification data

1.3.3.2. Standardized Data Serialization

All detection results are systematically transformed into standardized JSON objects. This approach ensures:

- Language-agnostic data representation
- Simplified integration across diverse module technologies
- Consistent data structure for reliable parsing by receiving components

1.3.3.3. Intelligent Data Routing

The API functions as a sophisticated message broker, routing detection data to appropriate destinations:

- Desktop application receives data for operator monitoring
- Mobile application receives alerts for management oversight
- Backend database receives comprehensive data for historical analysis

1.3.4. API.h Implementation

```
#ifndef SDBELT_API_H
#define SDBELT_API_H

#include <string>
#include <vector>
#include <memory>
#include <nlohmann/json.hpp>

namespace SDBelt {
namespace API {

// Structure to hold defect detection results from the YOLO model
struct DefectDetectionResult {
    std::string product_id;     // Unique identifier for the product
    std::string timestamp;      // Time when the detection occurred
    bool defect_present;        // Whether a defect was detected
    double confidence;    // Confidence score of the detection (0-1)

    // Bounding box coordinates for the detected defect
    struct {
```

```
        double x, y, width, height;
    } bbox;

    double defect_area_pct;   // Percentage of product area affected by defect
    std::string defect_type;  // Classification of the defect type
};

// API Service class that handles communication between YOLO module and other components
class APIService {
public:
    // Constructor with base URL for the backend endpoints
    APIService(const std::string& base_url = "http://localhost:8080");

    // Destructor to clean up resources
    ~APIService();

    // Sends a single defect detection result to the appropriate endpoints
    bool sendDetectionResult(const DefectDetectionResult& result);

    // Sends multiple detection results in a single batch request
    bool sendBatchResults(const std::vector<DefectDetectionResult>& results);

    // Reports system status data like CPU temperature and memory usage
    bool sendSystemStatus(double cpu_temp, double memory_usage, int camera_status);

    // Sends notifications or alerts to desktop and mobile applications
    bool sendNotification(const std::string& message, const std::string& severity);

private:
    // Helper method to send HTTP POST requests with JSON data
    bool sendPOSTRequest(const std::string& endpoint, const nlohmann::json& data);
```

```
    // Base URL for API endpoints
    std::string m_base_url;
};

// Utility function to get current timestamp in ISO 8601 format
std::string getCurrentTimestamp();

} // namespace API
} // namespace SDBelt
#endif // SDBELT_API_H
```

1.3.5. Using the API for Data Transmission

- **API Role:**
  A dedicated API (primarily in C++ with supportive tools) is responsible for handling data transmission. It acts as a central dispatcher, ensuring that all detection data is consistently forwarded to the designated endpoints.
- **Process Workflow:**
  - **Data Collection:**
    The image processing module captures real-time detection results.
  - **Data Serialization:**
    These results are organized into a JSON object.
  - **Data Transmission:**
    The JSON object is transmitted via an HTTP POST request using the API.
- **Endpoint Consumption:**
  The receiving modules, desktop app, mobile app, and backend systems, parse the JSON data to update dashboards, trigger notifications, or log the data for further analysis.

1.3.6. Implementation Details

```
// Example implementation of the sendDetectionResult method
bool APIService::sendDetectionResult(const DefectDetectionResult& result) {
    // Create JSON structure for detection result
```

```
    nlohmann::json json_data = {
        {"product_id", result.product_id},
        {"timestamp", result.timestamp},
        {"defect_present", result.defect_present},
        {"confidence", result.confidence},
        {"bbox", {
            {"x", result.bbox.x},
            {"y", result.bbox.y},
            {"width", result.bbox.width},
            {"height", result.bbox.height}
        }},
        {"defect_area_pct", result.defect_area_pct},
        {"defect_type", result.defect_type}
    };

    // Transmit data to the detection results endpoint
    return sendPOSTRequest("/api/detection_results", json_data);
}
```

This implementation demonstrates the data transformation process from structured C++ objects to standardized JSON format before transmission via HTTP POST requests.

1.3.7. System Integration Workflow

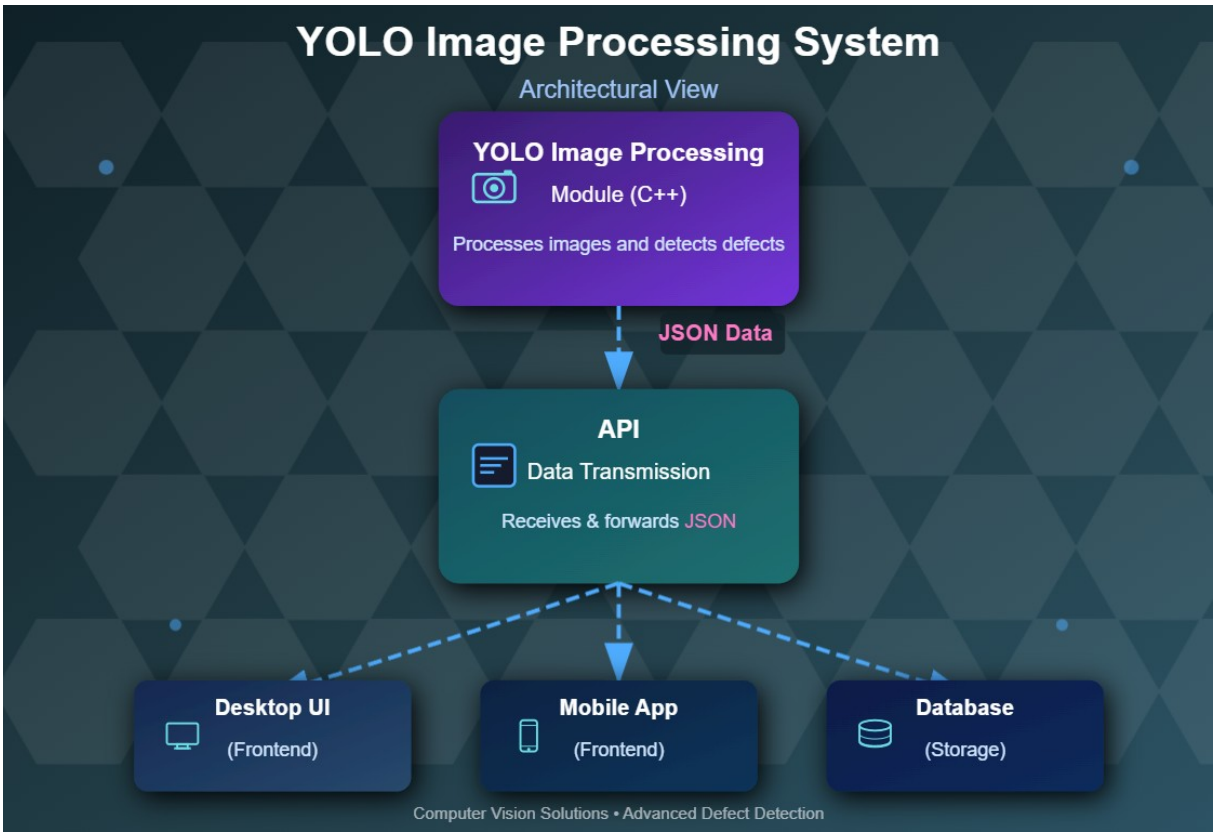
The API follows a precisely defined workflow to ensure reliable data transmission:

- **Real-time Data Acquisition**: The YOLO processing module generates detection results through image analysis.
- **Structured Data Conversion**: Raw detection data is organized into the DefectDetectionResult structure, ensuring all necessary information is captured.
- **JSON Serialization**: The C++ structure is converted to a standardized JSON object format.
- **Secure HTTP Transmission**: The JSON data is transmitted via POST requests to the appropriate endpoints.
- **Multi-destination Distribution**: Data is distributed to:

- Desktop application for real-time operator monitoring and intervention
- Mobile application for management alerts and remote monitoring
- Backend database for persistent storage, analytics, and reporting

1.3.8. Architectural Diagram

Below is a conceptual illustration of how the YOLO Image Processing module interacts with the API to distribute detection results to the desktop UI, mobile app, and database.



1.3.9. Enhanced Performance Considerations

Our implementation incorporates several advanced techniques to ensure optimal system performance:

- **Asynchronous Processing**: The API employs non-blocking operations to prevent the YOLO detection pipeline from being delayed by network operations.
- **Batch Processing Capabilities**: For high-throughput scenarios, the API supports batch transmission of multiple detection results, reducing network overhead.
- **Error Handling and Resilience**: Comprehensive error handling ensures system stability even during network disruptions or backend failures.
- **Cross-Platform Compatibility**: The implementation runs consistently across both Linux and Windows environments, providing deployment flexibility.

- **Performance Optimization**: The API minimizes overhead by maintaining persistent connections and optimizing data structures.

1.3.10. Security Considerations

The API implementation incorporates several security measures:

- **Data Validation**: All incoming and outgoing data is validated to prevent malformed requests.
- **Secure Transmission**: HTTPS can be enabled for encrypted data transmission.
- **Authentication Support**: The API supports token-based authentication for secure system access.

1.3.11. Conclusion

Implementing the API in this manner provides several key benefits:

- **Efficient Data Processing:**
  Real-time quality control is achieved through a high-speed pipeline from detection to endpoint communication.
- **Seamless Data Transmission:**
  By converting detection data into JSON, the API facilitates standardized communication across desktop, mobile, and backend modules.
- **Robust and Flexible Communication:**
  The overall structure supports real-time monitoring and logging, ensuring that defect detection data is promptly and reliably disseminated throughout the system.

1.4. Test Cases for YOLO-Based Image Processing Module

Feature	Purpose	Test Input / Action	Expected Result	Result
Fruit Detection	Verify that all fruits in a frame are	A single image containing 3	Exactly 5 bounding boxes	Planned

	correctly detected	apples and 2 oranges on a plain background	are returned (3 for apples, 2 for oranges), each with confidence ≥ 0.5	
Defect Identification	Ensure that known defects are flagged	An image of an apple with a brown bruise region, plus one unblemished apple	One defect bounding box around the bruise with class "defect" and confidence ≥ 0.6; the other apple has no defect box	Planned
No-False-Positive Detection	Confirm module does not falsely flag pristine items	An image of 5 flawless fruits	Zero "defect" boxes; exactly 5 "fruit" boxes	Planned
Multi-Camera Stream Handling	Validate that two camera feeds can be ingested and processed in parallel without dropping frames	Two simultaneous video streams (cam1.mp4 and cam2.mp4) each containing 10 frames of moving fruits	All 20 frames (10 per stream) processed; inference time per frame ≤ 200 ms; no dropped frames	Planned
Confidence Threshold Filtering	Test that detections below configured threshold are discarded	An image with one fruit detection at confidence=0.45 (below threshold T)	The low-confidence detection is filtered out; no bounding box returned	Planned
JSON Output & Schema Validation	Check that module outputs detection results in correct JSON format	Run detection on a test image and capture the module's JSON output	JSON contains keys: `product_id`, `timestamp`, `defect_present`, `confidence`, `bbox` (with 4 coords); types and nesting match the API spec	Planned
Real-Time Throughput	Verify pipeline meets minimum FPS requirement	Feed a 60-second 30 FPS test video to the module on Raspberry Pi	Sustained processing at ≥ 15 FPS over the entire video	Planned
Low-Light / High-Glare Robustness	Ensure detection still works under challenging lighting	A pair of test images of fruits under dim light (~50 lux) and under direct glare	Fruits still detected with confidence ≥ 0.5; defects (if present) detected with confidence	Planned

			≥ 0.6	
Error Handling: Missing Frame	Confirm module gracefully handles a dropped or empty frame	Inject a blank frame (all zeros) in the middle of a camera stream	Module logs a warning “Empty frame received” and skips inference without crashing; subsequent frames continue to process normally	Planned
Defect Area Calculation	Validate that post-processing correctly computes defect area percentage	An image where defect covers exactly 25% of the fruit bounding box	JSON field `defect_area_pct` reported as ~25.0 ± 1%; no other fields affected	Planned

2. Desktop Application Module

2.1. Overview

- The Desktop Application Module is intended to serve as the primary control interface for operators to interact with the SD Belt system in real time. As of the current stage of development, this module remains in the planning phase and has not yet been implemented.

However, it is designed to play a critical role in providing remote access and control over key system operations such as stopping or starting the belt, receiving system alerts, and maintaining log records of events.

- The application is planned to be developed using the C++ programming language in combination with the Qt framework, leveraging its graphical user interface capabilities as well as its built-in support for network communication. The user interface will be designed using Qt Widgets and will aim to provide a clear, functional, and user-friendly environment for industrial operators. The layout will include interactive controls and status displays that align with the workflow of the physical hardware system.
- Communication between the desktop application and the SD Belt system is expected to occur through a WebSocket-based messaging infrastructure. This approach allows for real-time, bidirectional data exchange between the desktop system and the Raspberry Pi. The messages exchanged will follow a structured JSON format to ensure clarity, extensibility, and machine-readability, particularly in future stages where multiple commands and status types may be introduced.
- Although no code has been implemented yet, several key functionalities are envisioned. These include the ability to send control commands such as stopping the belt, display real-time system notifications (e.g., error messages, operational status changes), and record user interactions and system events to a local log file. Furthermore, future versions of the application may support additional features such as camera switching (if multiple camera modules are integrated into the hardware) and system health monitoring, where diagnostic data from the Raspberry Pi could be periodically retrieved and displayed.
- At present, the module is still in the requirements and design discussion phase. No GUI elements or networking logic have been constructed. Implementation of this module is scheduled to begin after core hardware testing and initial integration with the Raspberry Pi are completed. Once development begins, detailed documentation covering source code structure, communication protocols, test cases, and user interaction flow will be prepared accordingly.

2.2. Implemented Features

Notify:

The module receives real-time system notifications (e.g., alerts, status changes, error messages) from the Raspberry Pi and forwards them to the GUI for immediate display. These notifications may also be forwarded to the mobile application used by supervisors. Notifications are sent with a 'targets' field to specify whether they should appear on desktop, mobile, or both.

Stop the Belt:

The user can stop the conveyor belt immediately through the GUI. This command is sent via WebSocket and is processed by the Raspberry Pi and relayed to the Arduino-based motor controller. The message includes the source to verify authority.

Logs:

The communication module is responsible for recording critical system events and data related to the product processing workflow. These records include received system responses, operator actions, notifications, error conditions, and product-specific processing outcomes. For each product, information such as whether an error occurred, and the corresponding error rate is logged. This enables monitoring of product performance and supports quality control processes. The logging

system is maintained in an extensible format and is suitable for future integration with category-specific analysis or reporting tools.

2.3. Possible Future Features

Camera Switching:

The communication layer may support commands for dynamically switching between different camera streams, allowing operators to monitor the system from multiple angles. However, camera video streaming will not be handled by WebSocket due to performance constraints. Instead, WebSocket will only send the switching command, while the stream itself will be handled via MJPEG over HTTP.

System Health Monitoring:

The desktop client may be extended to request periodic health data (e.g., CPU temperature, memory, hardware connection status) from the Raspberry Pi and display this information in a diagnostics panel.

2.4. Technologies Used

Component	Technology
Communication Protocol	WebSocket
C++ WebSocket Library	QtWebSockets
Logging	File-based (.txt or .json logs)
Programming Language	C++17

2.5. Message Format Examples

Stop Belt Command:

```
{
  "type": "command",
  "action": "stop_belt",
  "source": "desktop",
  "target": "raspberry",
  "timestamp": "2025-04-10T12:34:56"
}
```

Notification Message:

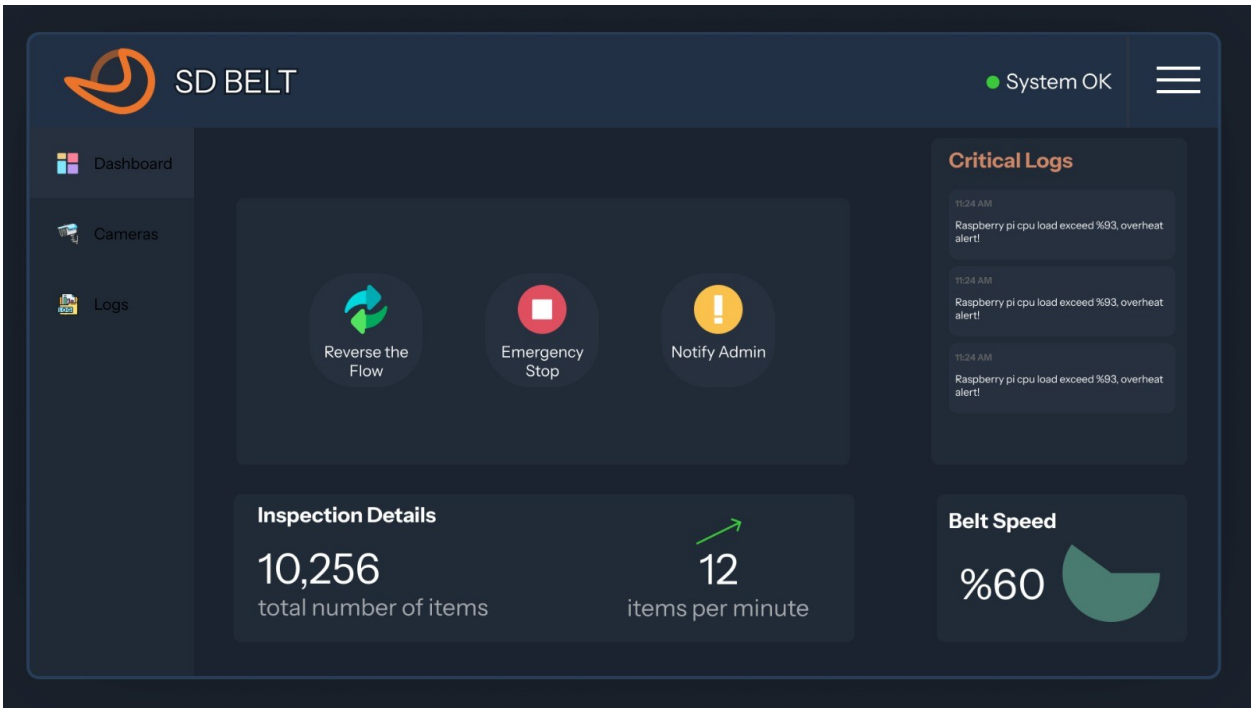
```
{
  "type": "notify",
  "message": "System overload detected",
  "severity": "critical",
  "targets": ["desktop", "mobile"],
  "timestamp": "2025-04-10T12:34:56"
```

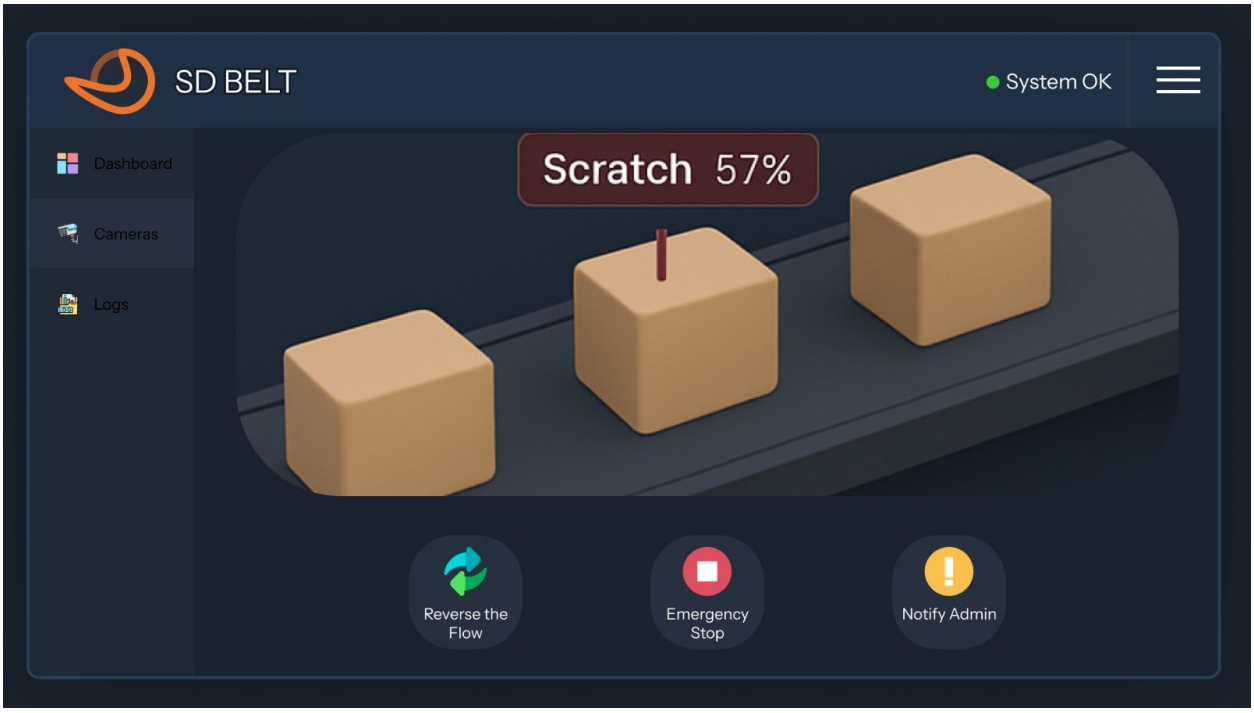
}

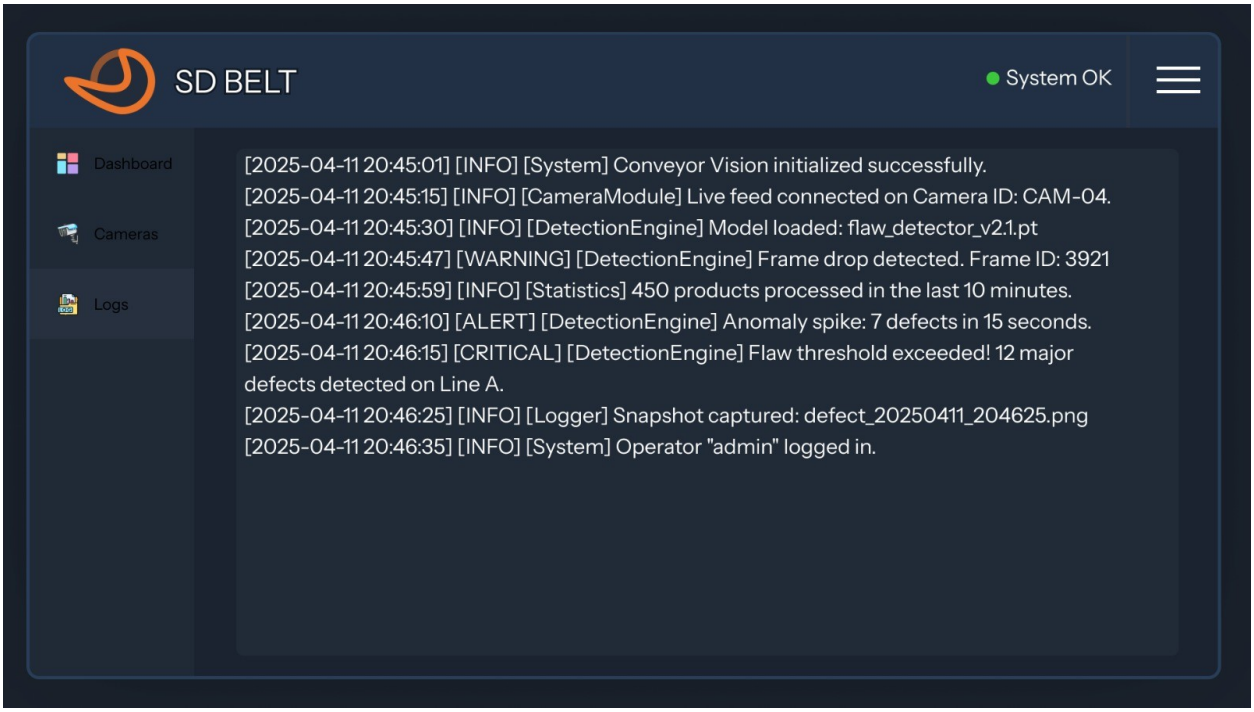
Switch Camera Command:

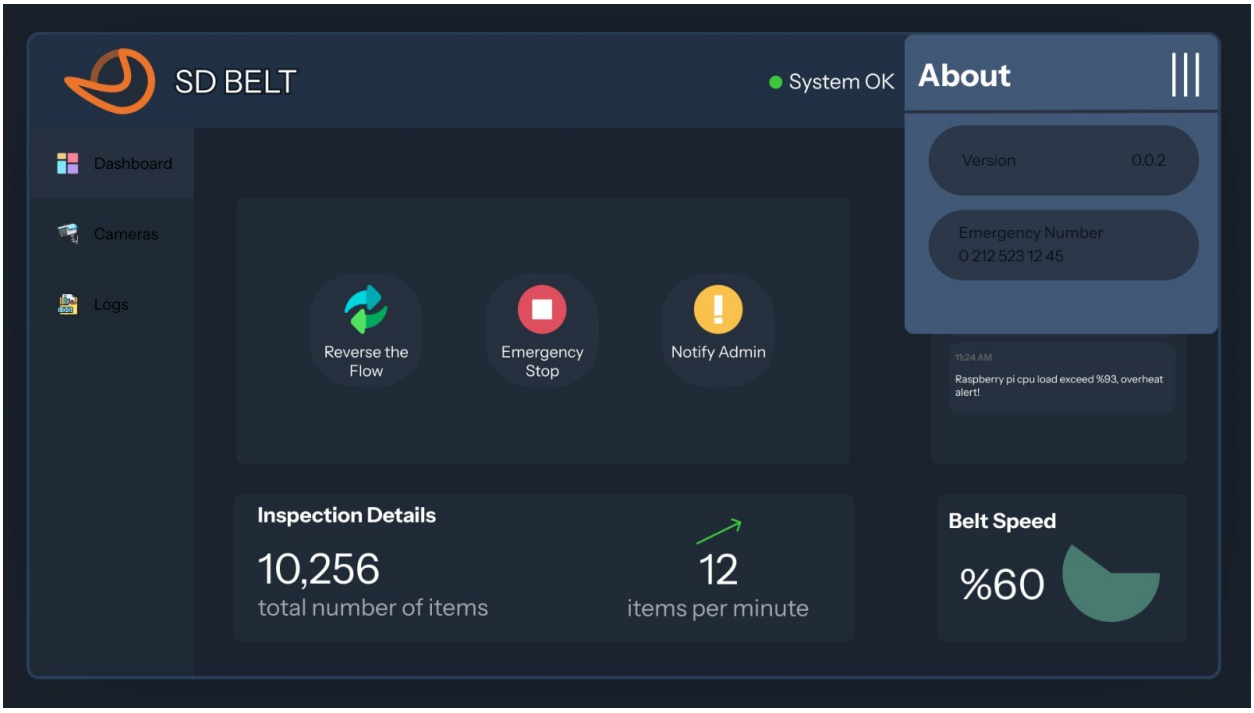
```
{
  "type": "command",
  "action": "switch_camera",
  "camera_id": "cam2"
}
```

2.6. Desktop Figma Designs




SD BELT
System OK
Dashboard
Cameras
Logs
Scratch 57%
Reverse the Flow
Emergency Stop
Notify Admin

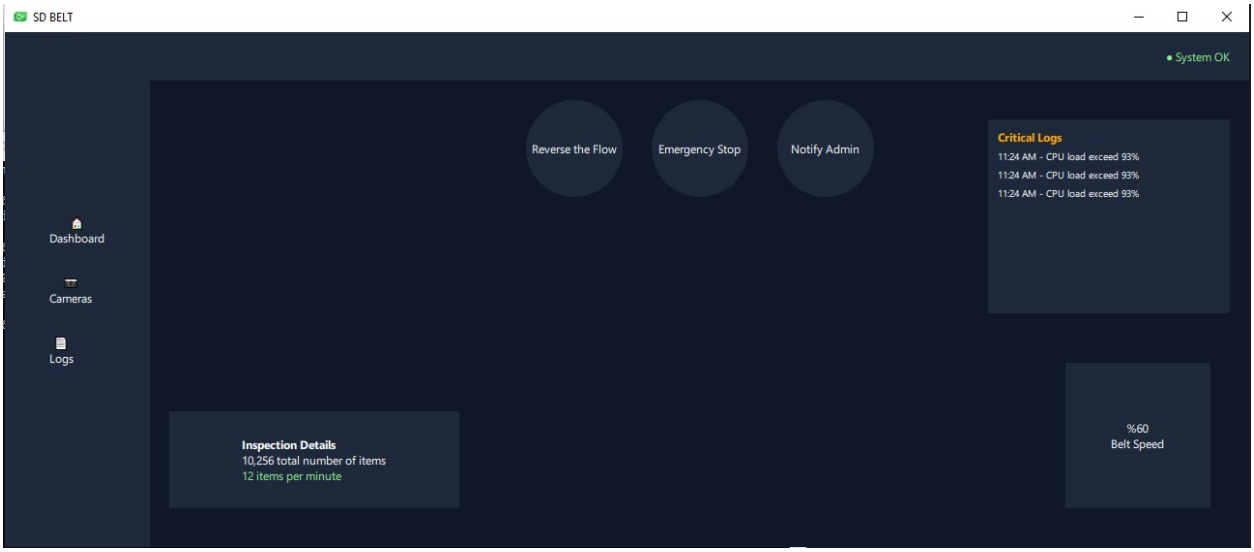

SD BELT
System OK
Dashboard
Cameras
Logs
[2025-04-11 20:45:01] [INFO] [System] Conveyor Vision initialized successfully.
[2025-04-11 20:45:15] [INFO] [CameraModule] Live feed connected on Camera ID: CAM-04.
[2025-04-11 20:45:30] [INFO] [DetectionEngine] Model loaded: flaw_detector_v2.1.pt
[2025-04-11 20:45:47] [WARNING] [DetectionEngine] Frame drop detected. Frame ID: 3921
[2025-04-11 20:45:59] [INFO] [Statistics] 450 products processed in the last 10 minutes.
[2025-04-11 20:46:10] [ALERT] [DetectionEngine] Anomaly spike: 7 defects in 15 seconds.
[2025-04-11 20:46:15] [CRITICAL] [DetectionEngine] Flaw threshold exceeded! 12 major defects detected on Line A.
[2025-04-11 20:46:25] [INFO] [Logger] Snapshot captured: defect_20250411_204625.png
[2025-04-11 20:46:35] [INFO] [System] Operator "admin" logged in.



2.7. Implementation Details

QT Framework

GUI(Work in Progress):



QML:

```
import QtQuick 2.15
import QtQuick.Controls 2.15

ApplicationWindow {
    visible: true
    width: 1200
    height: 700
    title: "SD BELT"

    Rectangle {
        anchors.fill: parent
        color: "#0f172a"

        // Sidebar
        Rectangle {
            width: 150
            anchors.top: parent.top
            anchors.bottom: parent.bottom
            color: "#1e293b"

            Column {
                spacing: 30
                anchors.centerIn: parent

                Text { text: "🏠\nDashboard"; color: "white"; horizontalAlignment: Text.AlignHCenter }
                Text { text: "📷\nCameras"; color: "white"; horizontalAlignment: Text.AlignHCenter }
                Text { text: "📄\nLogs"; color: "white"; horizontalAlignment: Text.AlignHCenter }
            }
        }

        // Top Bar
        Rectangle {
            height: 50
            anchors.left: parent.left
            anchors.right: parent.right
            anchors.top: parent.top
            anchors.leftMargin: 150
            color: "#1e293b"
```

```
            Text {
                text: "\u25CF System OK"
                color: "lightgreen"
                anchors.verticalCenter: parent.verticalCenter
                anchors.right: parent.right
                anchors.rightMargin: 20
            }
        }

        // Main Content
        Rectangle {
            anchors.top: parent.top
            anchors.left: parent.left
            anchors.margins: 20
            anchors.topMargin: 70
            anchors.leftMargin: 170
            anchors.right: parent.right
            anchors.bottom: parent.bottom
            color: "#0f172a"

            // Control Buttons
            Row {
                spacing: 30
                anchors.top: parent.top
                anchors.horizontalCenter: parent.horizontalCenter

                Repeater {
                    model: ["Reverse the Flow", "Emergency Stop", "Notify Admin"]
                    delegate: Rectangle {
                        width: 100; height: 100
                        radius: 50
                        color: "#1e293b"

                        Text {
                            anchors.centerIn: parent
                            text: modelData
```

```
            // Inspection Details
            Rectangle {
                width: 300; height: 100
                color: "#1e293b"
                anchors.left: parent.left
                anchors.bottom: parent.bottom
                anchors.leftMargin: 0
                anchors.bottomMargin: 20

                Column {
                    anchors.centerIn: parent
                    Text { text: "Inspection Details"; color: "white"; font.bold: true }
                    Text { text: "10,256 total number of items"; color: "white" }
                    Text { text: "12 items per minute"; color: "lightgreen" }
                }
            }

            // Belt Speed
            Rectangle {
                width: 150; height: 150
                color: "#1e293b"
                anchors.right: parent.right
                anchors.bottom: parent.bottom
                anchors.bottomMargin: 20
                anchors.rightMargin: 20

                Text {
                    anchors.centerIn: parent
                    text: "%60\nBelt Speed"
                    color: "white"
                    horizontalAlignment: Text.AlignHCenter
                }
            }
        }
    }
}
```

WebSocket Communications:

QTClientWebsocket.cpp > ...

```cpp
#include <QApplication>
#include <QPushButton>
#include <QVBoxLayout>
#include <QLineEdit>
#include <QTextEdit>
#include <QWebSocket>
#include <QDebug>

class WebSocketClientWidget : public QWidget
{
    Q_OBJECT

public:
    WebSocketClientWidget(const QUrl &url, QWidget *parent = nullptr) : QWidget(parent), socket()
    {
        QVBoxLayout *layout = new QVBoxLayout(this);

        messages = new QTextEdit(this);
        messages->setReadOnly(true);
        layout->addWidget(messages);

        input = new QLineEdit(this);
        layout->addWidget(input);

        QPushButton *sendButton = new QPushButton("Send", this);
        layout->addWidget(sendButton);

        connect(&socket, &QWebSocket::connected, this, &WebSocketClientWidget::onConnected);
        connect(&socket, &QWebSocket::textMessageReceived, this, &WebSocketClientWidget::onMessageReceived);
        connect(sendButton, &QPushButton::clicked, this, &WebSocketClientWidget::sendMessage);

        socket.open(url);
    }

private slots:
    void onConnected()
    {
        messages->append("Connected to server.");
    }
```

```
    void onMessageReceived(QString message)
    {
        messages->append("Server: " + message);
    }

    void sendMessage()
    {
        QString msg = input->text();
        if (!msg.isEmpty())
        {
            socket.sendTextMessage(msg);
            messages->append("You: " + msg);
            input->clear();
        }
    }

private:
    QWebSocket socket;
    QTextEdit *messages;
    QLineEdit *input;
};

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    QUrl url(QStringLiteral("ws://localhost:12345"));  // Our Local Server, dummy address for now
    WebSocketClientWidget client(url);
    client.show();

    return app.exec();
}
```

2.8. Test Cases for Desktop Application Module

Feature	Purpose	Input	Expected Result	Result
Notify	Verify that system notifications sent from Raspberry Pi are displayed on the desktop GUI and optionally forwarded to mobile clients.	{ "type": "notify", "message": "Test Alert", "severity": "info", "targets": ["desktop", "mobile"] }	Notification is shown on desktop GUI and logged properly.	Planned
Stop the Belt	Test whether the user can stop the SD Belt via GUI and if the command is relayed through Raspberry Pi to the motor.	{ "type": "command", "action": "stop_belt", "source": "desktop", "target": "raspberry" }	The belt stops within 2 seconds and confirmation appears in GUI or logs.	Planned

Logs	Ensure that all key actions and events are being written to the log file in a consistent format.	Trigger a stop command and receive a notification.	Both events are recorded with timestamps in the log file.	Planned

Switch Camera	Check if the system can switch active camera streams when commanded from GUI.	{
		"type": "command",
		"action": "switch_camera",
		"camera_id": "cam2"
		}
			The stream switches to cam2 and GUI reflects the new feed.	Planned

System Health Monitoring	Validate that Raspberry Pi sends health metrics and GUI displays them correctly.	{
		"type": "status",
		"cpu_temp": 61.2,
		"memory_usage": 45.5
		}
			CPU temperature and memory usage are updated in the GUI panel.	Planned

Basic GUI Interaction with QT	Operating with interactive GUI elements such as (buttons, labels, input fields).	Verify that the user can interact with basic GUI components and see immediate visual feedback.	The GUI layout remains responsive and delivers the expected actions with visual feedbacks.	Planned
Proper Layering of QT Widgets	Making sure of UI experience is flawless and clean	Verify that layers on top of each other acts as expected based on input.	Where Pointer(Mouse) is interacted on screen will get effected by input only.	Planned

Measure WebSocket Communication Latency	WebSocket ping-pong latency check	User clicks a “Test Latency” button
		{
		"type": "ping", "timestamp": <current_time>
		}
			Server replies with a pong message
			({ "type": "pong", "timestamp": <received_time> }), around 50 ms latency expected
				Planned

Utilizing Qt for WebSocket Communication	To leverage Qt's native WebSocket support for efficient and seamless communication in a C++ desktop application.	Need for asynchronous message handling without blocking the UI.	Asynchronous data exchange handled smoothly via Qt's event-driven architecture.	Planned

3. Hardware Module

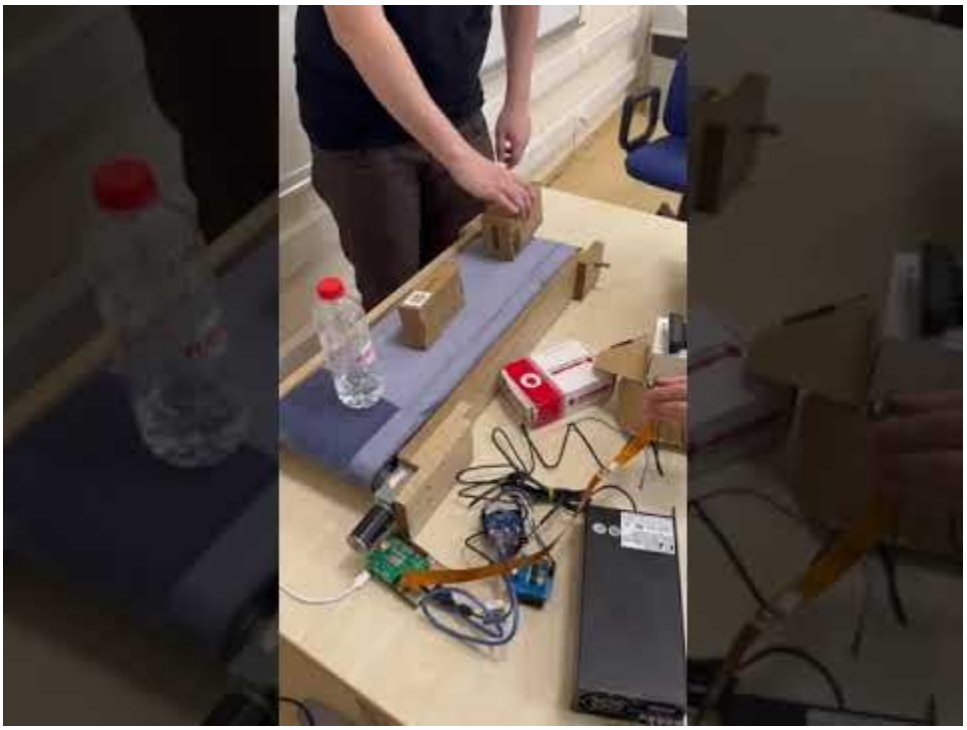
3.1. Overview

- This module covers the physical infrastructure of the system built for transporting and routing products. It includes the SD Belt system, motor control components, power supply infrastructure, and hardware elements that are planned to be integrated in the future, such as vision and sorting mechanisms.
- The SD Belt is driven by a 24V DC motor, which is controlled via a motor driver board. This motor driver is connected to an Arduino microcontroller, which generates the necessary digital signals to operate the motor. The Arduino is responsible for executing direct motor control tasks such as starting and stopping the belt.
- A serial communication interface has been established so that the Arduino can receive external commands physically. During initial testing, the motor was successfully started and stopped. The motor and driver system have proven capable of driving the SD Belt reliably. A 24V DC power supply is used to power the motor, and a regulated 5V output is included to support lower-voltage components when needed.
- The physical assembly of the SD Belt has been completed. The motor mounting, belt alignment, and product path have been structurally finalized and verified. Test runs have shown that products can move smoothly across the belt, and the mechanical balance of the system is stable. The frame has been designed with scalability in mind, allowing room for additional components.
- Several hardware extensions are planned for future implementation. Most notably, 2 to 4 camera modules will be mounted above the belt to allow for visual inspection of the products from multiple angles. Once installed, these cameras will be used to capture live images of moving products, forming the basis for potential image-processing-based classification tasks.
- Additionally, a sorting mechanism will be added at the end of the SD Belt. This mechanism will consist of a directional arm mounted on a servo motor, which will be responsible for diverting products to different sides based on their evaluation result. The servo motor will be controlled by the Arduino and will receive angle adjustment commands depending on the classification outcome. The planned servo model is the MG996R, a high-torque servo capable of delivering up to 11 kg·cm at 6V with a rotation range of approximately 0° to 180°. Thanks to its strong torque and metal gear construction, the MG996R is suitable for operating a lightweight but rigid sorting arm. The arm is expected to be made of wood, which offers a practical balance between weight, stiffness, and ease of construction, allowing it to physically push products to the correct direction during the sorting process without compromising speed or precision.
- In conclusion, the current hardware module is fully operational for product transport and is mechanically complete. Camera modules and a servo-driven directional arm are planned additions. The system has been designed to accommodate these components and is structurally ready for future expansions.

Servo Motor



3.2. SD Belt Videos







3.3. Implementation Overview

The system consists of two main components: an Arduino sketch and a Raspberry Pi C++ program that communicate via serial port.

Arduino Side: The Arduino controls a BTS7960 motor driver that powers a conveyor belt motor. It accepts commands through serial communication to adjust speed, change direction, and perform various stop operations. Key features include:

- Percentage-based speed control (0-100%)
- Gradual acceleration and deceleration capability
- Immediate and gradual stop options
- Direction control (forward/reverse)

Raspberry Pi Side: The Raspberry Pi runs a C++ program that sends commands to the Arduino through serial communication. The implementation uses:

- A dedicated `ArduinoSerial` class to handle all serial communication
- Simple command interface with methods like `setSpeed()`, `stopImmediate()`, etc.
- A main program that demonstrates a specific conveyor belt operation sequence

The communication protocol between devices is text-based with commands like:

- `PCT:X` - Set speed percentage to X (0-100)
- `STOP:0` - Immediate stop
- `STOP:X` - Gradual stop with ramp rate X
- `START:X` - Start with acceleration rate X
- `DIR:X` - Set direction (1=forward, -1=reverse)
- `REV` - Toggle direction

This implementation provides a robust and flexible way to control the conveyor belt speed and direction while allowing for smooth acceleration and deceleration, which is crucial for reliable operation in a production environment.

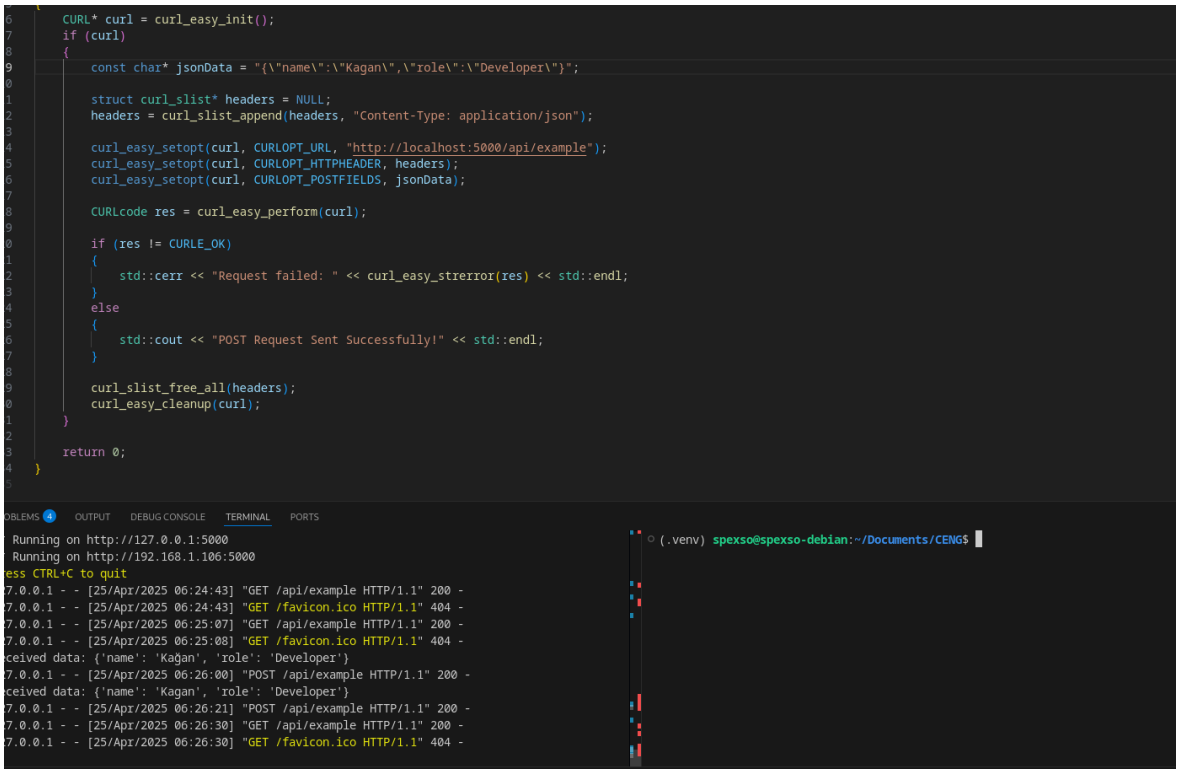
Arduino Implementation

```
// Key definitions in Arduino code
const int RPWM = 9;    // PWM signal for clockwise rotation (forward)
const int LPWM = 10;   // PWM signal for counter-clockwise rotation (reverse)
const int R_EN = 7;    // Enable pin for clockwise rotation
const int L_EN = 8;    // Enable pin for counter-clockwise rotation
// Motor control variables
int currentSpeed = 0;       // Current speed (0-255)
int targetSpeed = 0;        // Target speed for acceleration/deceleration
int currentDirection = 1;   // 1 for forward, -1 for reverse
int rampRate = 5;           // Speed change per iteration during ramping
```

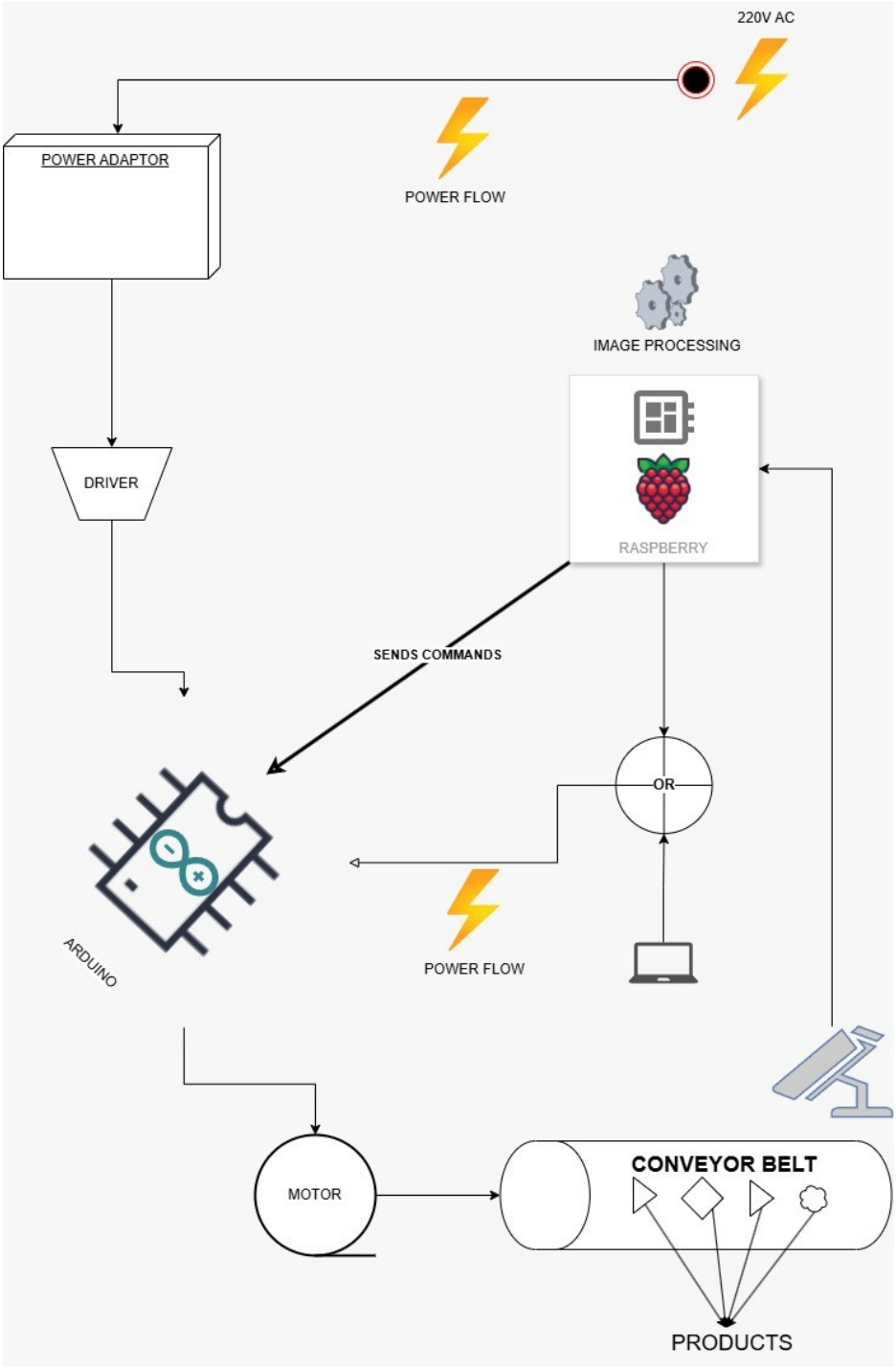
Raspberry Pi Implementations

```
// ArduinoSerial class definition
class ArduinoSerial {
public:
    ArduinoSerial(const std::string& portName);
    ~ArduinoSerial();
    std::string sendCommand(const std::string& command);
    bool isConnected() const;
    std::string setSpeed(int percent);
    std::string stopImmediate();
    std::string stopGradual(int rampRate);
    std::string start(int rampRate = 5);
    std::string setDirection(int direction);
    std::string reverseDirection();
};
```

Example Setup



3.4. SD Belt Workflow



3.5. Test Cases for Hardware Module

Feature	Purpose	Test Input / Action	Expected Result	Result
SD Belt Motor Activation	Verify that the motor responds correctly to control signals from Arduino.	Power on system and send "start" command from Raspberry Pi to Arduino.	SD Belt begins moving smoothly and at consistent speed.	Passed
Motor Stop Command	Ensure that the belt stops when a "stop" command is sent.	Send stop command from Raspberry Pi to Arduino while belt is in motion.	SD Belt stops within 2 seconds.	Passed
Power Supply Check	Validate that the power supply provides stable 24V under load.	Measure voltage across motor terminals while the SD Belt is running.	Voltage remains within 23–25V during operation.	Passed
Arduino-Pi Connection	Ensure serial connection between Raspberry Pi and Arduino is active.	Disconnect and reconnect the serial cable and observe command transmission.	Motor responds again after reconnection.	Passed
SD Belt Frame Stability	Confirm that mechanical frame holds the system steady.	Run the belt at full speed with a standard load placed on it.	No wobble or excessive vibration observed.	Passed
Camera Mounting Plan	Plan and verify suitable positions for multiple camera modules.	Attach mock-up cameras or placeholders at designated positions above the belt.	At least 3 camera viewpoints offer full coverage of product angles.	Planned
Camera Wiring Plan	Ensure power and data paths for camera modules are feasible.	Plan wiring paths from Pi to 3-4 camera positions without overlap or signal interference.	Cabling layout is clean and Raspberry Pi ports are reachable for each camera.	Planned
Raspberry Pi - Arduino Serial	Verify that motor control	Send start/stop commands over	Motor responds	Passed

Communication	commands are reliably transmitted from Raspberry Pi to Arduino via serial interface.	serial and observe Arduino motor response.	accordingly to the commands without delay.	
Product Transport by SD Belt	Confirm that the SD Belt can transport sample products without obstruction or slippage.	Place a sample product on the belt and activate motion.	Product is carried to the other end without misalignment or falling.	Passed
Camera-Based Product Capture	Verify that the camera records passing products during belt movement for future processing.	Activate camera while the belt is moving with sample products.	Camera captures visual data correctly during motion.	Planned
Webcam Fallback Test	Ensure that a USB webcam can be used as a backup in case of Raspberry Pi camera module failure.	Disconnect Pi camera and connect USB webcam, then run camera test application.	Video feed is displayed and functions equivalently to Pi camera module.	Planned
Servo Motor Sorting Test	Ensure the servo motor correctly moves the wooden arm to sort products based on commands.	Send sort-left and sort-right commands to the Arduino and observe servo movement.	Servo rotates arm in correct direction and pushes product accordingly.	Planned

4. Mobile Application Module

4.1. Introduction

The SD Belt mobile application is designed to provide remote monitoring and management capabilities for production line administrators and managers. This application will enable users to monitor defect detection processes and adjust system parameters from anywhere, enhancing operational efficiency and quality control.

4.2. Technology Approach

4.2.1. Technology Stack

For the SD Belt mobile application, we have selected the following technology stack:

- **Framework**: Qt6 with QML
- **Programming Language**: C++, QML(UI)
- **Design Tools**: Figma for comprehensive UI/UX prototyping
- **Target Platforms**: Android and iOS

4.2.2. Reasons for Choosing Qt and C++

The decision to utilize Qt with C++ was made based on several factors:

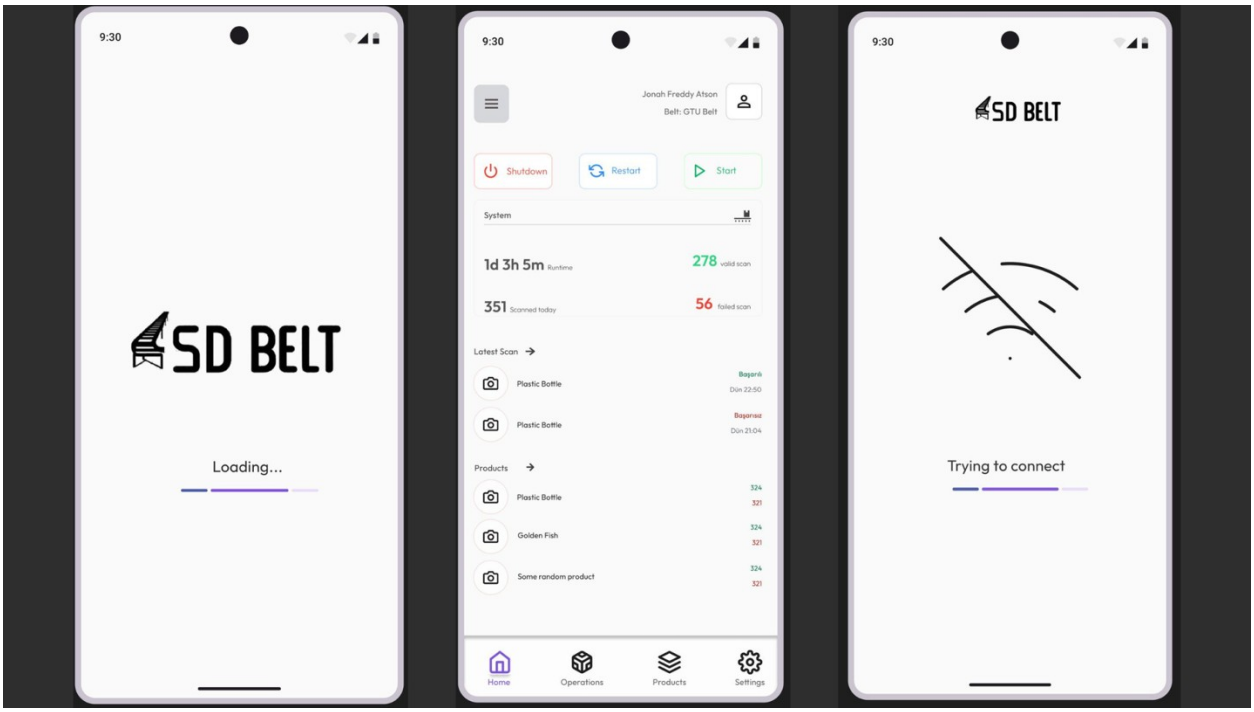
1. **Cross-platform capabilities**: Qt allows us to develop once and deploy on both Android and iOS platforms, significantly reducing development time and maintenance overhead.
2. **C++ Focused development**: Since at least 70% of the codebase is required to be in C/C++, Qt provides an ideal solution that meets our project requirements while maintaining development efficiency.

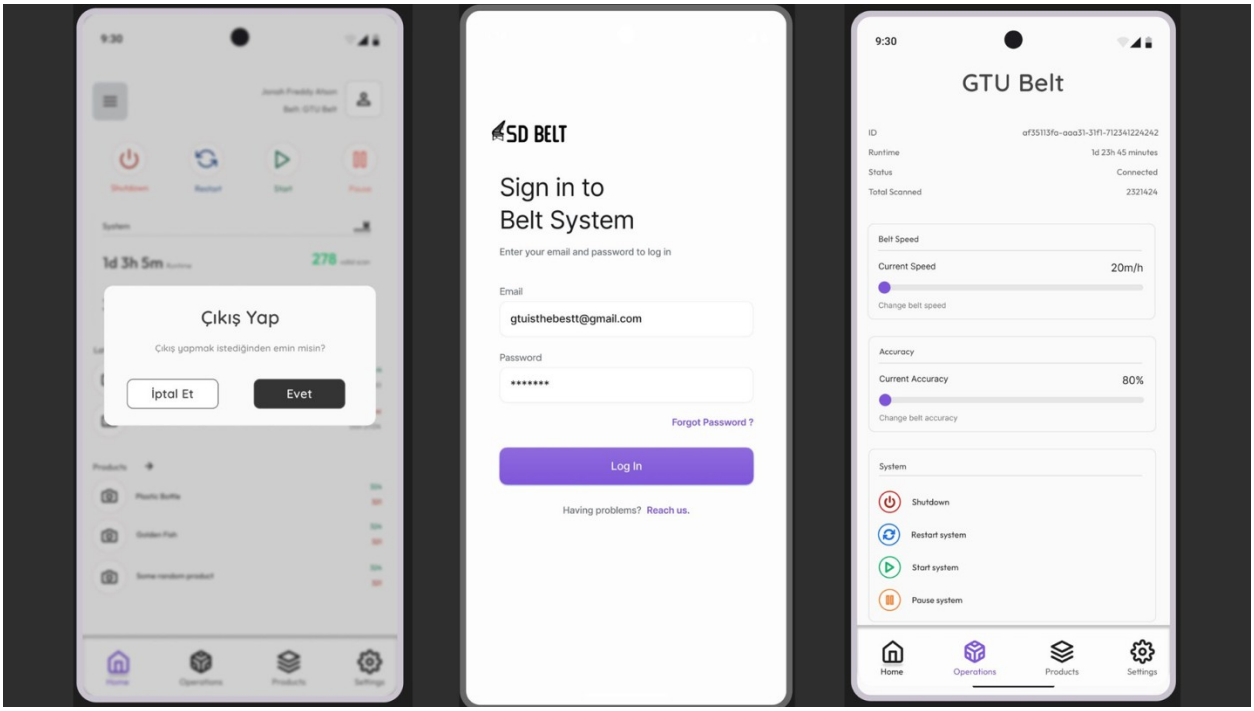
4.3. Current Progress

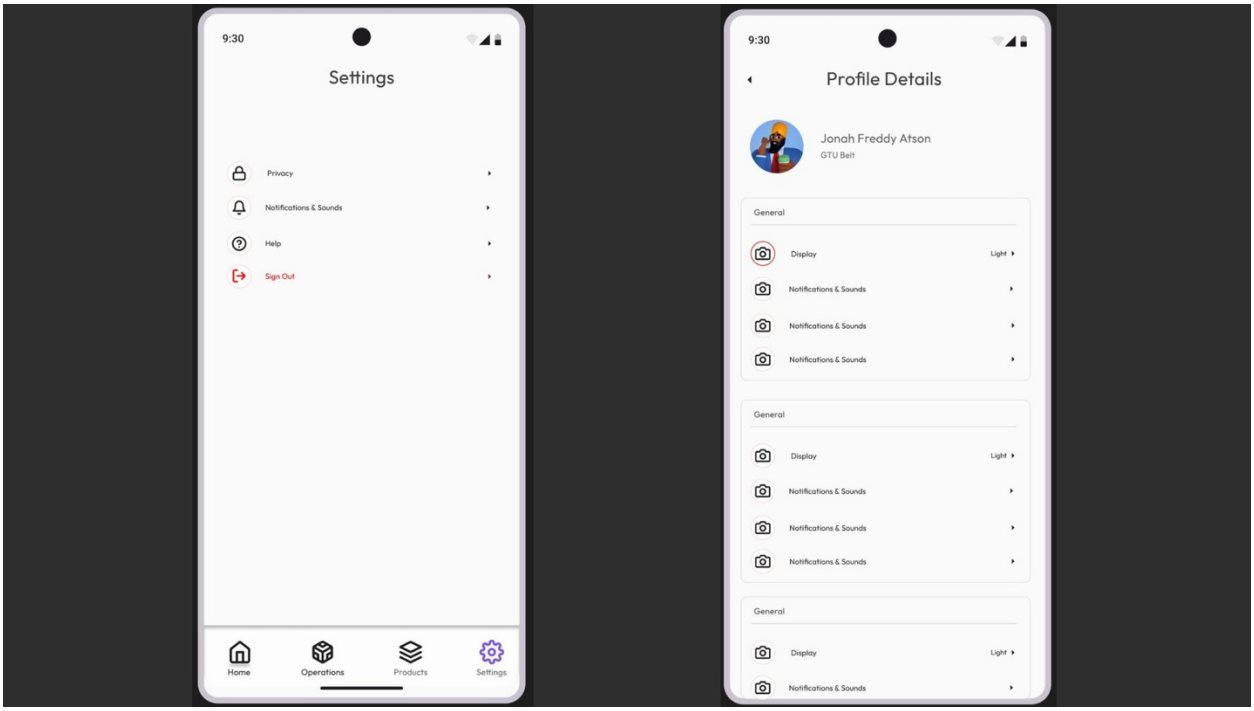
4.3.1. Implementation Phase

- Following the completion of the design and planning stages, we have transitioned into the implementation phase of the SD Belt mobile application. This phase focuses on transforming the designs and planned software architecture into a functional, working product.

- We have initiated the development of core system modules, including the user authentication system (UserService), belt control and configuration logic (BeltService), and production statistics retrieval (StatisticService). Each of these service modules is under implementation using C++ to align with the project’s technical requirements and to maximize performance and maintainability.
- Simultaneously, the user interface is being constructed using QML, following the navigation structure and visual guidelines established during the design phase. Key screens such as the login interface, real-time dashboard, profile management, and settings page are under active development. Essential UI components like buttons, cards, lists, and tab navigation are being developed as reusable and scalable elements to support the application's future growth.
- The mobile application’s architecture allowing for smooth future integration with backend services via RESTful APIs.
- At this stage, both backend logic and frontend presentation layers are under parallel development. Core functionality is gradually being integrated and tested to ensure alignment with project goals. The implementation phase is iterative, allowing for continuous feedback, adjustment, and refinement as necessary.







- **Design System**: Developed a consistent design language including color schemes, typography, component styles, and interactive elements to ensure visual coherence across the application.

4.3.2. Development Planning

We are currently in the planning phase for implementation:

- **Technology Research**: Completed preliminary research on Qt framework and its capabilities for mobile development.
- **Development Environment Setup**: Researching the necessary tools and configurations for Qt with C++ development for cross-platform mobile applications.
- **Architecture Planning**: Designing the software architecture that will best implement our Figma designs using Qt's capabilities.
- **Learning Process**: Since our team is not extensively familiar with Qt, we have allocated time for research and skill development to ensure a smoother development experience.

4.4. Development Roadmap

4.4.1. Phase 1: Setup and Foundation

- Configure Qt development environment for cross-platform mobile development
- Set up version control and project structure
- Create basic application skeleton with navigation framework
- Design additional pages in Figma for remaining features (real-time camera monitoring interface, etc.)
- Implement application theme and style guidelines based on our existing Figma designs

4.4.2. Phase 2: Core UI Implementation

- Develop QML components for all designed screens
- Implement the bottom tab navigation system
- Create UI components (cards, buttons, inputs, etc.)
- Implement screen transitions and basic UI flow

4.4.3. Phase 3: Functionality and Logic

- Develop authentication and user management logic
- Implement API communication layer for backend integration
- Create data models for representing production information
- Develop real-time monitoring functionality

4.4.4. Phase 4: Testing and Refinement

- Cross-platform testing on Android and iOS devices
- Bug fixing and interface refinement

4.5. Integration with Other Modules

The mobile application integrates with other SD Belt modules through:

- **Backend API Integration**: The backend module will serve as the central bridge between the physical belt system and the mobile application, facilitating all data exchange. It will handle RESTful API calls for data retrieval, system configuration, and monitoring communication, ensuring a seamless flow of information across the system.

4.6. Implementation Details

The mobile application is designed to offer monitoring, control, and analytics for the SD Belt. Currently the system follows a modular architecture where responsibilities are distributed among three primary services: UserService, StatisticService, and BeltService. Each service exposes core functionalities through QML-invokable methods, allowing for seamless interaction with the QML frontend.

The **UserService** handles user authentication and profile management. The login(const QString &username, const QString &password) function sends a POST request to the backend to validate the provided credentials. On successful login, it returns a response (token & success message) that allows the app to grant access. The getUserDetails()function fetches user-related data such as name, email, and profile photo—used to personalize the app interface.

The **StatisticService** exposes metrics from the belt system. The getBeltRuntime() method returns a formatted string representing the total operational time (e.g., "1d 2h 30m"). This allows the user to track machine uptime. The getTotalScanned(), getPassedCount(), and getFailedCount() functions provide integer values representing the number of items processed, passed, and failed respectively—useful for evaluating production quality. Additionally, getItemBreakdown() returns a structured list of each item type, along with its corresponding pass/fail statistics.

The **BeltService** provides hardware-level control of the conveyor belt system.
The startBelt() and restartBelt()functions send control commands to initiate or reinitialize the belt's operation. The shutdownBelt() safely halts the system, essential for maintenance or in case of emergencies. The setBeltSpeed(int speed) function allows the user to configure the speed percentage dynamically, with values typically ranging from 0 to 100. Similarly, setBeltAccuracy(int accuracy) adjusts the accuracy level of the scanning or detection module, allowing trade-offs between speed and precision.

On the frontend, QML is used to build an interactive and visually clean user interface. Button actions will be linked to service functions using signals and slots, enabling users to control the belt, view statistics, and manage their accounts in real time.

Belt_Service.h

Belt_Service.h

```h
#ifndef BELT_SERVICE_H
#define BELT_SERVICE_H

#include <QObject>

/**
 * @brief The BeltService class provides control and configuration for the conveyor belt system.
 *
 * Includes methods to control belt operations and configure hardware settings.
 */
class BeltService : public QObject {
    Q_OBJECT
public:
    explicit BeltService(QObject *parent = nullptr);

    /**
     * @brief Sends a command to start the belt.
     */
    Q_INVOKABLE void startBelt();

    /**
     * @brief Sends a command to restart the belt operation.
     */
    Q_INVOKABLE void restartBelt();

    /**
     * @brief Sends a command to safely shut down the belt.
     */
    Q_INVOKABLE void shutdownBelt();

    /**
     * @brief Sets the operational speed of the belt.
     * @param speed The speed level (e.g., 0-100).
     */
    Q_INVOKABLE void setBeltSpeed(int speed);

    /**
     * @brief Adjusts the accuracy setting of the scanning or detection system.
     * @param accuracy The desired accuracy percentage (e.g., 90 for 90%).
     */
    Q_INVOKABLE void setBeltAccuracy(int accuracy);
};

#endif // BELT_SERVICE_H

```

Belt_Service.cpp

Belt_Service.cpp

```cpp
#include "belt_service.h"
#include <QNetworkAccessManager>
#include <QNetworkRequest>
#include <QNetworkReply>
#include <QUrl>
#include <QJsonDocument>
#include <QJsonObject>
#include <QEventLoop>
#include <QDebug>

BeltService::BeltService(QObject *parent)
    : QObject(parent)
{
    networkManager = new QNetworkAccessManager(this);
}

void BeltService::startBelt() {
    sendPostRequest("http://localhost:1616/api/v1/system/start");
}

void BeltService::restartBelt() {
    sendPostRequest("http://localhost:1616/api/v1/system/restart");
}

void BeltService::shutdownBelt() {
    sendPostRequest("http://localhost:1616/api/v1/system/shutdown");
}

QString BeltService::getCameraFootageUrl() const {
    return "http://localhost:1616/api/v1/system/stream";
}

void BeltService::setBeltSpeed(int speed) {
    QUrl url("http://localhost:1616/api/v1/system/speed");
    QNetworkRequest request(url);
    request.setHeader(QNetworkRequest::ContentTypeHeader, "application/json");

    QJsonObject json;
    json["speed"] = speed;

    QNetworkReply *reply = networkManager->post(request, QJsonDocument(json).toJson());
    QEventLoop loop;
    QObject::connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
    loop.exec();

    if (reply->error() == QNetworkReply::NoError) {
        qDebug() << "[BeltService] Speed updated to" << speed;
    } else {
        qDebug() << "[BeltService] Speed update failed:" << reply->errorString();
    }
    reply->deleteLater();
}

void BeltService::setBeltAccuracy(int accuracy) {
    QUrl url("http://localhost:1616/api/v1/system/accuracy");
    QNetworkRequest request(url);
    request.setHeader(QNetworkRequest::ContentTypeHeader, "application/json");

    QJsonObject json;
    json["accuracy"] = accuracy;

    QNetworkReply *reply = networkManager->post(request, QJsonDocument(json).toJson());
    QEventLoop loop;
    QObject::connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
    loop.exec();

    if (reply->error() == QNetworkReply::NoError) {
        qDebug() << "[BeltService] Accuracy updated to" << accuracy;
    } else {
        qDebug() << "[BeltService] Accuracy update failed:" << reply->errorString();
    }
    reply->deleteLater();
}

void BeltService::sendPostRequest(const QString &urlString) const {
    QUrl url(urlString);
    QNetworkRequest request(url);
    request.setHeader(QNetworkRequest::ContentTypeHeader, "application/json");

    QNetworkReply *reply = networkManager->post(request, QByteArray());
    QEventLoop loop;
    QObject::connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
    loop.exec();

    if (reply->error() == QNetworkReply::NoError) {
        qDebug() << "[BeltService] Request to" << urlString << "successful.";
    } else {
        qDebug() << "[BeltService] Request to" << urlString << "failed:" << reply->errorString();
    }
    reply->deleteLater();
}

```

Statistic_Service.h

Statistic_Service.h

```h
#ifndef STATISTIC_SERVICE_H
#define STATISTIC_SERVICE_H

#include <QObject>
#include <QVariant>

/**
 * @brief The StatisticService class provides access to belt operation statistics.
 *
 * This includes runtime durations, item counts, and detailed breakdowns.
 */
class StatisticService : public QObject {
    Q_OBJECT
public:
    explicit StatisticService(QObject *parent = nullptr);

    /**
     * @brief Retrieves the current runtime of the belt system.
     * @return A formatted string showing total operation time (e.g., "1d 2h 15m").
     */
    Q_INVOKABLE QString getBeltRuntime() const;

    /**
     * @brief Gets the total number of scanned items.
     * @return Integer count of scanned items.
     */
    Q_INVOKABLE int getTotalScanned() const;

    /**
     * @brief Gets the count of passed items.
     * @return Integer count of successfully passed items.
     */
    Q_INVOKABLE int getPassedCount() const;

    /**
     * @brief Gets the count of failed items.
     * @return Integer count of items that failed quality control.
     */
    Q_INVOKABLE int getFailedCount() const;

    /**
     * @brief Provides a breakdown per item type with pass/fail statistics.
     * @return A QVariantList of item summary maps (name, passed, failed).
     */
    Q_INVOKABLE QVariantList getItemBreakdown() const;
};

#endif // STATISTIC_SERVICE_H

```

User_Service.h

User_Service.h

```h
#ifndef USER_SERVICE_H
#define USER_SERVICE_H

#include <QObject>
#include <QString>
#include <QJsonObject>
#include <QNetworkAccessManager>

/**
 * @brief The UserService class handles user authentication and profile retrieval via HTTP backend.
 */
class UserService : public QObject {
    Q_OBJECT
public:
    explicit UserService(QObject *parent = nullptr);

    /**
     * @brief Confirms the user's login credentials via POST request to the backend.
     * @param username The username provided by the user.
     * @param password The password provided by the user.
     * @return True if login is successful, false otherwise.
     */
    Q_INVOKABLE QString login(const QString &username, const QString &password) const;

    /**
     * @brief Fetches the user's profile information from the backend.
     * @param username The username for which to fetch the profile.
     * @return A QJsonObject containing fields like name, email, photo URL, etc.
     */
    Q_INVOKABLE QJsonObject getUserDetails() const;

private:
    QNetworkAccessManager *networkManager; ///< Used for handling HTTP requests.
};

#endif // USER_SERVICE_H

```

QML

```qml
Component {
    id: homePage
    Rectangle {
        color: "#fefefe"
        anchors.fill: parent

        ColumnLayout {
            anchors.fill: parent
            anchors.margins: 20
            spacing: 16

            Text {
                text: "SD Belt Control Panel"
                font.pixelSize: 20
                font.bold: true
                horizontalAlignment: Text.AlignHCenter
                Layout.alignment: Qt.AlignHCenter
            }

            RowLayout {
                spacing: 12
                Button { text: "Start"; Layout.fillWidth: true; onClicked: beltService.startBelt() }
                Button { text: "Restart"; Layout.fillWidth: true; onClicked: beltService.restartBelt() }
                Button { text: "Shutdown"; Layout.fillWidth: true; onClicked: beltService.shutdownBelt() }
            }

            Rectangle {
                width: parent.width
                height: 2
                color: "#c0c0c0"
            }

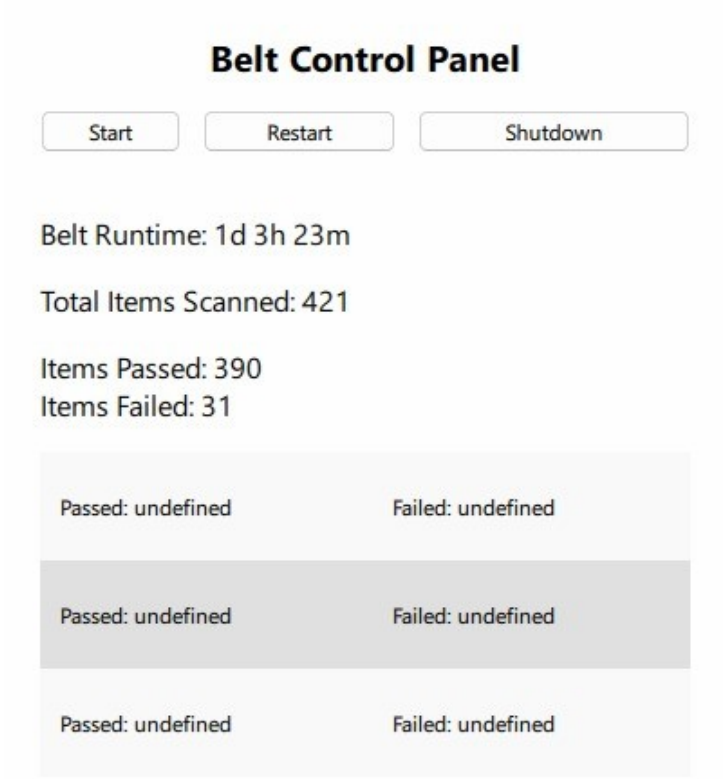
            Text {
                text: "SD Belt Runtime: " + statisticService.getBeltRuntime()
                font.pixelSize: 16
            }

            Text {
                text: "Total Items Scanned: " + statisticService.getTotalScanned()
                font.pixelSize: 16
            }

            Text {
                text: "Items Passed: " + statisticService.getPassedCount() + "\nItems Failed: " + statisticService.getFailedCount()
                font.pixelSize: 16
            }

            ListView {
                Layout.fillHeight: true
                Layout.fillWidth: true
                model: statisticService.getItemBreakdown()
                delegate: Rectangle {
                    height: 60
                    width: parent.width
                    color: index % 2 === 0 ? "#f9f9f9" : "#e0e0e0"
                    RowLayout {
                        anchors.fill: parent
                        spacing: 10
                        Text { text: model.name; Layout.fillWidth: true }
                        Text { text: "Passed: " + model.passed; Layout.fillWidth: true }
                        Text { text: "Failed: " + model.failed; Layout.fillWidth: true }
                    }
                }
            }
        }
    }
}
```

GUI





4.7. Test Cases for Mobile Application Module

Feature	Purpose	Test Input / Action	Expected Result	Result
User Login	Verify login with credentials	Enter valid username and password	User is redirected to dashboard; token/session is created	Planned
Invalid Login	Prevent login with invalid credentials	Enter wrong password or username	Error message shown: “Invalid credentials”	Planned
Daily Scan	Validate correct	Backend sends	App shows: 8 valid, 2	Planned

Statistics	rendering of product scan statistics	data: total: 10, valid: 8	invalid, ratio 80% valid	
Belt Accuracy Display	Ensure belt accuracy is correctly calculated and shown	Backend sends accuracy = 80%	App displays “Belt Accuracy: 80%” with correct formatting	Planned
Belt Speed Visualization	Confirm belt speed updates correctly from backend	Speed rate = 3 sent from backend	App shows “Belt Speed Rate = 3” in the stats panel	Planned
Offline Mode (No Internet)	Check app response when device is offline	Turn off internet; open app	App displays “Offline - Connect to sync latest stats” message	Planned
Belt Start Command	Ensure belt starts when command is triggered from UI	Tap “Start Belt” button in control panel	Belt start command sent; UI reflects “Belt Running”	Planned
Real-Time Throughput	Verify pipeline sends live info	Backend constantly sends latest info	App updates display accordingly without needing manual refresh	Planned
Runtime Statistics Fetch	Validate belt runtime display	Call getBeltRuntime() after operation	App displays runtime string (e.g., “0d 3h 40m”)	Planned

5. Database and Backend Module

5.1. Introduction

The SD Belt backend is designed as a robust and secure central communication and data management hub that connects the mobile, desktop, and hardware modules. It facilitates efficient data transfer, system integration, and real-time interactions across the entire SD Belt project. The backend module ensures high-performance processing, reliable data management, and coherent operation across all interconnected components.

5.2. Technology Approach

5.2.1. Technology Stack The SD Belt backend will utilize the following technology stack:

- Framework: Spring Boot
- Programming Language: Java
- Database: PostgreSQL
- Communication Protocol: WebSockets, RESTful APIs
- Cloud Services: AWS or Azure for deployment, scalability, and security

5.2.2. Reasons for Choosing Spring Boot and PostgreSQL

1. Comprehensive Ecosystem: Spring Boot's extensive features, including built-in security, configuration management, and powerful integration capabilities, accelerate the development process.
2. Scalable and Maintainable: Ideal for microservice architectures, Spring Boot allows the application to scale efficiently while remaining maintainable.
3. Reliability and Performance: PostgreSQL offers strong data consistency, ACID compliance, and performance tuning capabilities necessary for complex data transactions and real-time analytics.

5.3. Backend Architecture

5.3.1. Database Design The database schema will include:

- Product Details: Product identifiers, manufacturing data, and specifications.
- Quality Metrics: Real-time quality checks, defect detection results, and evaluation data.
- User Authentication: Secure management of user data, credentials, roles, and permissions.
- Historical Analytics: Long-term storage of analytics data to track production efficiency and defect patterns.

5.3.2. API and Communication Layer

- RESTful APIs: Designed for CRUD operations and interactions with mobile, desktop, and hardware modules, including authentication, data retrieval, and hardware command interfaces.
- WebSocket Integration: Implemented for real-time streaming of camera footage and sensor data to mobile and desktop applications.
- Messaging Queue: Utilized for asynchronous processing of high-volume data exchanges, ensuring stable system performance.

5.3.3. Infrastructure Planning

- Cloud Infrastructure: Cloud-based hosting on AWS or Azure, providing robust scaling capabilities, redundancy, and security.

- Continuous Integration and Continuous Delivery (CI/CD): Automated deployment pipelines established for rapid and reliable updates.

5.4. Development Roadmap

5.4.1. Phase 1: Infrastructure and Database Setup

- Docker-based environment setup for consistent development and production environments.
- PostgreSQL deployment and initial database schema configuration.
- Implementation of automated version control and CI/CD pipelines.

5.4.2. Phase 2: Core Backend Development

- Secure authentication and authorization systems using Spring Security.
- RESTful APIs development for handling CRUD operations and data integration.
- Middleware implementation for data validation, structured logging, and comprehensive error handling.

5.4.3. Phase 3: Real-time Integration with Modules

- Implement camera data streaming using WebSocket for seamless real-time video feeds to mobile and desktop clients.
- Develop APIs to handle control commands sent from the mobile and desktop apps to the hardware module (e.g., adjusting belt speed, stopping the belt).
- Ensure efficient, real-time bidirectional communication between backend, mobile, desktop, and hardware.

5.4.4. Phase 4: Testing and Optimization

- Perform extensive integration and unit testing across all API endpoints and real-time streams.
- Optimize performance for high responsiveness, minimizing latency in data transmission and processing.
- Conduct rigorous security assessments, penetration testing, and adherence to security best practices.

5.5. Integration with Other Modules The backend coordinates interactions across all SD Belt modules:

- Hardware Module: Processes and forwards camera data to other modules and receives operational commands from desktop and mobile apps.
- Mobile Application: Receives real-time camera streams and sends control commands for the hardware.
- Desktop Application: Receives real-time camera streams, provides analytics capabilities, and sends control commands to manage hardware operations effectively.

Through a well-structured architecture, robust APIs, and optimized real-time communication channels, the backend ensures efficient, secure, and reliable integration, significantly enhancing the operational effectiveness of the overall SD Belt system.

5.6. Implementation Overview

The backend module forms the central hub of the SD Belt system, managing communication, data, and system logic across hardware, mobile, and desktop interfaces. It is built using the Spring Boot framework and backed by a PostgreSQL database.

Core Features Include:

• Secure user authentication and role-based access control
• RESTful APIs for mobile, desktop, and hardware interaction
• WebSocket-based real-time streaming for camera and sensor data
• Comprehensive database schema supporting product tracking, quality metrics, and analytics

Technology Highlights:

• Spring Boot: Accelerates development with built-in features like dependency injection, security, and easy configuration
• PostgreSQL: Strong consistency, and powerful query capabilities

Communication Protocols and Interfaces:

• HTTP/HTTPS for standard API interactions
• WebSocket for real time data feeds
• Command structure over REST for issuing hardware control signals (e.g., /api/system/stop, /api/system/speed/{value})

This backend implementation ensures smooth real-time control and monitoring, reliable data handling.

ScanController.java

```java
@RestController
@RequiredArgsConstructor
@RequestMapping("/api/v1")
public class ScanController {

    private final ScanService service;

    @Operation(summary = "Get all scans", description = "Get all scans")
    @ApiResponses(value = {
            @ApiResponse(responseCode = "200", description = "Scans retrieved successfully"),
            @ApiResponse(responseCode = "400", description = "Invalid request. Check the provided scan ID."),
            @ApiResponse(responseCode = "401", description = "Unauthorized request. Please check your credentials."),
            @ApiResponse(responseCode = "500", description = "Internal server error. Unable to retrieve scans.")
    })
    @GetMapping("/scans")
    public Response<List<Scan>> getAllScans(@ParameterObject @ModelAttribute ScanFilter filter) {
        List<Scan> scans = service.find(filter);
        return ResponseBuilder.build(status:200, scans);
    }

    @Operation(summary = "Add scan", description = "Add a new scan")
    @ApiResponses(value = {
            @ApiResponse(responseCode = "200", description = "Scan added successfully"),
            @ApiResponse(responseCode = "400", description = "Invalid request. Check the provided scan details."),
            @ApiResponse(responseCode = "401", description = "Unauthorized request. Please check your credentials."),
            @ApiResponse(responseCode = "500", description = "Internal server error. Unable to add scan.")
    })
    @PostMapping("/scans")
    public Response<Scan> addScan(@RequestBody Scan scan) {
        service.create(scan.productId(), scan.isSuccess(), scan.errorMessage());
        return ResponseBuilder.build(status:200, scan);
    }

    @Operation(summary = "Get general statistics", description = "Get general statistics")
    @ApiResponses(value = {
            @ApiResponse(responseCode = "200", description = "Statistics retrieved successfully"),
            @ApiResponse(responseCode = "401", description = "Unauthorized request. Please check your credentials."),
            @ApiResponse(responseCode = "500", description = "Internal server error. Unable to retrieve statistics.")
    })
    @GetMapping(name = "/scans/statistics", params = {"startDate", "endDate"})
    public Response<GeneralStatistics> getGeneralStatistics(@RequestParam LocalDateTime startDate, @RequestParam LocalDateTime endDate) {
        GeneralStatistics statistics = service.generateStatistics(startDate, endDate);
        return ResponseBuilder.build(status:200, statistics);
    }

    @Operation(summary = "Get product statistics", description = "Get product statistics")
    @ApiResponses(value = {
            @ApiResponse(responseCode = "200", description = "Statistics retrieved successfully"),
            @ApiResponse(responseCode = "400", description = "Invalid request. Check the provided product ID."),
            @ApiResponse(responseCode = "401", description = "Unauthorized request. Please check your credentials."),
            @ApiResponse(responseCode = "500", description = "Internal server error. Unable to retrieve statistics.")
    })
    @GetMapping(value = "/scans/statistics", params = {"productId", "startDate", "endDate"})
    public Response<ProductStatistics> getProductStatistics(@RequestParam(required = true) UUID productId, @RequestParam LocalDateTime startDate, @RequestParam LocalDateTime endDate) {
        ProductStatistics statistics = service.generateStatistics(productId, startDate, endDate);
        return ResponseBuilder.build(status:200, statistics);
    }
}
```

ScanService.java

```java
/**
 * Service interface for managing scan records of products in the conveyor belt scanning system.
 * <p>
 * Provides methods for creating scan entries and retrieving scan histories,
 * supporting both full listing and filtered queries based on scan parameters.
 */
public interface ScanService {

    /**
     * Creates a new scan record for a product.
     *
     * @param productId    the UUID of the product that was scanned; must not be null
     * @param isSuccess    flag indicating whether the scan was successful; must not be null
     * @param errorMessage an optional error message; should be null or empty if {@code isSuccess} is true
     * @throws IllegalArgumentException if the product ID is invalid or missing required data
     */
    void create(UUID productId, Boolean isSuccess, String errorMessage);

    /**
     * Retrieves a list of all scan records in the system.
     *
     * @return a list of all {@link Scan} entries; never null, may be empty
     */
    List<Scan> list();

    /**
     * Finds scan records matching the provided filter criteria.
     *
     * @param filter a {@link ScanFilter} object containing search conditions such as date ranges,
     *               product IDs, or success status; must not be null
     * @return a list of scan records that match the filter; never null, may be empty
     */
    List<Scan> find(ScanFilter filter);


    /**
    * Generates general statistics for all scan records within the specified time range.
    *
    * @param startDate the beginning of the time range (inclusive); must not be null
    * @param endDate   the end of the time range (inclusive); must not be null and not before {@code startDate}
    * @return a {@link GeneralStatistics} object containing aggregated scan data (e.g., total scans, success/failure rate)
    * @throws IllegalArgumentException if the date range is invalid or any parameter is null
    */
    GeneralStatistics generateStatistics(LocalDateTime startDate, LocalDateTime endDate);

    /**
     * Generates statistics for a specific product within the specified time range.
     *
     * @param productId  the UUID of the product to generate statistics for; must not be null
     * @param startDate  the beginning of the time range (inclusive); must not be null
     * @param endDate    the end of the time range (inclusive); must not be null and not before {@code startDate}
     * @return a {@link ProductStatistics} object with metrics like scan count, error rate, and last scan timestamp
     * @throws IllegalArgumentException if any input is null or if the date range is invalid
     */
    ProductStatistics generateStatistics(UUID productId, LocalDateTime startDate, LocalDateTime endDate);
}
```

ProductService.java

```java
/**
 * Service interface for managing product-related operations in the conveyor belt scanning system.
 * <p>
 * This service provides CRUD operations to handle product metadata which may be scanned and logged
 * during the belt process. Implementations of this interface are responsible for interacting with
 * the persistence layer or invoking domain logic.
 */
public interface ProductService {

    /**
     * Creates a new product entry with the given name and description.
     *
     * @param name        the name of the product; must not be null or empty
     * @param description a brief description of the product; can be null or empty
     */
    void create(String name, String description);

    /**
     * Updates an existing product identified by the given ID.
     *
     * @param id          the UUID of the product to update; must not be null
     * @param name        the new name of the product; must not be null or empty
     * @param description the new description of the product; can be null or empty
     * @throws IllegalArgumentException if the product does not exist
     */
    void update(UUID id, String name, String description);

    /**
     * Deletes the product identified by the given ID.
     *
     * @param id the UUID of the product to delete; must not be null
     * @throws IllegalArgumentException if the product does not exist
     */
    void delete(UUID id);

    /**
     * Retrieves the product with the specified ID.
     *
     * @param id the UUID of the product; must not be null
     * @return the product entity
     * @throws IllegalArgumentException if the product does not exist
     */
    Product get(UUID id);

    /**
     * Attempts to find a product with the specified ID.
     *
     * @param id the UUID of the product; must not be null
     * @return an {@link Optional} containing the product if found, or empty if not
     */
    Optional<Product> find(UUID id);

    /**
     * Lists all available products in the system.
     *
     * @return a list of all product entities; never null
     */
    List<Product> list();
}
```

SystemService.java

```java
/**
 * Service interface for managing the overall conveyor belt scanning system.
 * <p>
 * Provides lifecycle operations such as start, stop, and restart, as well as
 * configuration updates and status retrieval.
 */
public interface SystemService {

    /**
     * Retrieves the current status and configuration of the scanning system.
     *
     * @return the current {@link System} object containing system metadata and state
     */
    System get();

    /**
     * Starts the scanning system.
     * <p>
     * This operation typically initializes all components and begins processing
     * products on the conveyor belt.
     */
    void start();

    /**
     * Stops the scanning system.
     * <p>
     * This operation halts all active scanning and processing, effectively putting
     * the system into a paused or offline state.
     */
    void stop();

    /**
     * Restarts the scanning system.
     * <p>
     * Equivalent to calling {@link #stop()} followed by {@link #start()},
     * used to reinitialize system components or recover from errors.
     */
    void restart();

    /**
     * Updates the system's configuration or metadata, such as its name or description.
     *
     * @param name        the new name for the system
     * @param description the new description for the system
     */
    void update(String name, String description);
}
```

5.7. Test Cases for Database and Backend Module

Feature	Purpose	Test Input / Action	Expected Result	Result
User Authentication	Verify login and token creation	Send valid username and password to `/api/auth/login`	API returns status 200 with a valid JWT token in response	Planned
Invalid Login Handling	Prevent login with incorrect credentials	Send wrong password to `/api/auth/login`	API returns status 401 with error message "Invalid credentials"	Planned
Product Data Insertion	Ensure product defect info is correctly stored	Send POST request with product data (timestamp, defect flag etc.)	Data is stored in database's corresponding table	Planned
Real-Time Data Broadcast	Validate WebSocket sends real-time updates	Trigger a product scan event	Subscribed clients (mobile/desktop) receive WebSocket message	Planned
Database Constraint Check	Validate rejection of invalid data formats	Send malformed data (e.g., missing `timestamp`)	API returns 400 Bad Request; no insertion into DB	Planned
Command Handling	Confirm backend processes control commands	Send an action for controlling the belt	Command is forwarded to hardware module and acknowledged	Planned
Invalid API Endpoint	Test server response to invalid route	Send GET to a not existing endpoint	Returns 404 Not Found with a proper error message	Planned
Unauthorized Access	Only authorized users can invoke endpoints	Send a request to the protected route	Returns 401 Unauthorized with a proper error message	Planned
List Products	All products are listed	Send list products	Returns list of products	Planned

		request		